

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ ĐIỆN TỬ



Họ và Tên	MSSV
Tô Minh Cường	22661861
Nguyễn Công Luận	22646121
Nguyễn Tiến Quân	2265692

**ĐỀ TÀI: ĐO KHOẢNG CÁCH BẰNG CẢM BIẾN SIÊU ÂM
US-100 BẰNG RASPBERRT PI4**

Chuyên ngành: Dự án kỹ thuật nâng cao
Mã chuyên ngành: 422001504301

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

THÔNG TIN CHUNG

Họ và tên sinh viên: Tô Minh Cường MSHV:22661861

Lớp :DHDTMT18ATT Khóa:18.....

Chuyên ngành :Kĩ thuật máy tính Mã chuyên ngành: 4220015043

SĐT :0364327891

Email :tominhcuong2041505@gmail.com

Địa chỉ liên hệ : huyện bình chánh, Thành phố Hồ Chí Minh
.....

Tên đề tài : đo khoảng cách bằng cảm biến US-100 và RP2040
.....
.....

Người hướng dẫn : Th.S Nguyễn Duy Khanh

SĐT :

Email :

Cơ quan công tác :

Họ và tên sinh viên: Nguyễn Công Luận MSHV: 22646121

Lớp : DHDTMT18ATT Khóa: 18.....

Chuyên ngành : Kỹ thuật máy tính..... Mã chuyên ngành: 4220015043

SĐT : 0984035721

Email : ncongluan118@gmail.com

Địa chỉ liên hệ : Gò Vấp, Thành phố Hồ Chí
Minh

Tên đề tài : đo khoảng cách bằng cảm biến US-100 và RP2040.....

.....

.....

Người hướng dẫn : Th.S Nguyễn Duy Khanh

SĐT :

Email :

Cơ quan công tác :

Họ và tên sinh viên: Nguyễn Tiến Quân..... MSHV:22656921

Lớp :DHDTMT18ATT Khóa:K18

Chuyên ngành :Kỹ thuật máy tính Mã chuyên ngành: 4220015043

SĐT :0907976102

Email :tienquan0807@gmail.com.....

Địa chỉ liên hệ :Huyện Nhà bè, Thành phố Hồ Chí Minh

Tên đề tài : đo khoảng cách bằng cảm biến US-100 và RP2040

.....
.....

Người hướng dẫn : Th.S Nguyễn Duy Khanh

SĐT :

Email :

Cơ quan công tác :

Tp. Hồ Chí Minh, ngày tháng năm 20.....

Người hướng dẫn
(Ký và ghi rõ họ tên)

Sinh viên
(Ký và ghi rõ họ tên)

MỤC LỤC

DANH MỤC HÌNH ẢNH	8
DANH MỤC BẢNG BIỂU	Error! Bookmark not defined.
DANH MỤC TỪ VIẾT TẮT.....	10
MỞ ĐẦU.....	11
CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI.....	13
1.1 Bối cảnh và lý do chọn đề tài	13
1.2 Tổng quan các nghiên cứu liên quan.....	13
1.2.1 Nhóm nghiên cứu về cảm biến và thu thập dữ liệu	13
1.2.2 Nhóm nghiên cứu về xử lý tín hiệu và thuật toán.....	14
1.2.3 Nhóm nghiên cứu về giao diện người dùng và hiển thị.....	14
1.3 Những hạn chế hiện tại và vấn đề đề tài cần giải quyết.....	14
1.4 Mục tiêu cải tiến của đề tài.....	15
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	16
2.1 Nguyên lý hoạt động của linh kiện.....	16
2.1.1 Cảm biến siêu âm US-100	16
2.1.2 Vi điều khiển RP2040.....	17
2.1.3 Opto cách ly	19
2.2 Nguyên lý giao thức truyền nhận dữ liệu UART	20
2.2.1 Cấu trúc truyền UART.....	20
2.2.2 Nguyên lý hoạt động.....	21
2.2.3 Ứng dụng trong đề tài	21
2.2.4 Kỹ thuật Software UART (Bit-Banging).....	21
2.3 Nguyên lý mạch đo khoảng cách	22
2.4 Thuật toán và phương pháp xử lý tín hiệu	22
2.4.1 Thuật toán đo khoảng cách	22
2.4.2 Phương pháp xử lý nhiễu	22
2.4.3 Phương pháp hiển thị dữ liệu	23

2.5	Thuật toán CRC	23
2.5.1	Khái niệm	23
2.5.2	Công thức CRC sử dụng trong đề tài	23
2.5.3	Thuật toán CRC (diễn giải)	24
2.6	Các nội dung lý thuyết liên quan khác	24
2.6.1	Giao diện Visual C#	24
2.6.2	Chip chuyển đổi CH340G	24
2.6.3	Tiêu chuẩn thiết kế PCB (IPC-2221)	24
2.6.4	Nguyên lý Ổn áp Zener (Zener Regulator)	25
CHƯƠNG 3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	26
3.1	Tính toán các giá trị cho schematic	26
3.2	Tính toán cho PCB	26
3.2.1	Khoảng cách (Clearance) trong thiết kế PCB	26
3.2.2	Quy tắc Lỗ khoan và Vòng đồng (Routing Vias & Annular Ring)	30
3.2.3	Polygon Pour (Vùng Đồng Đồng)	31
3.3	Tính toán các giá trị theo các công thức lý thuyết	33
3.3.1	Tính toán Khối Nguồn (U1: AMS1117-5.0)	33
3.3.2	Mạch Zener ($R1 = 220 \Omega \rightarrow Z1 = 3.3 \text{ V}$) —3V3_LAP	33
3.3.3	CH340G (USB $\leftrightarrow$ UART)	34
3.3.4	Phân tích và Tính toán Khối Cách ly Tín hiệu (U5, U6 – Optocoupler 6N137) 35	
3.4	Thiết kế bo mạch	36
3.5	Thiết kế mô hình	38
3.5.1	Lớp giao diện	38
3.5.2	Lớp Giao tiếp và Cách ly (Communication & Isolation Layer)	38
3.5.3	Lớp Điều khiển (Control Layer)	39
3.6	Thiết kế các giải thuật cho code	39
3.7	Thiết kế giao diện phần mềm (GUI)	46
3.7.1	Mục tiêu thiết kế	46
3.7.2	Bố cục giao diện	46

3.7.3	Mình họa giao diện phần mềm.....	47
3.7.4	Luồng hoạt động (Sequence Diagram)	47
CHƯƠNG 4 THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ.....		49
4.1	Phân tích tín hiệu Soft-UART (Giao tiếp PC) bằng Oscilloscope	49
4.1.1	Mức logic 0 / Logic 1	49
4.1.2	Tốc độ Baudrate	50
4.1.3	Timing (Cạnh xung) và Xung nhiễu	51
4.2	Phân tích tín hiệu UART (Giao tiếp Cảm biến) bằng Oscilloscope	51
4.3	Đo điện áp hoạt động bằng VOM	52
4.4	Kiểm tra chức năng Cảnh báo LED	54
4.5	Đánh giá chung và Kết quả thực tế	54
4.5.1	Kết quả kiểm tra (Test) 1000 gói tin.....	55
4.5.2	Kết quả đo khoảng cách (Chế độ READ).....	56
4.5.3	Đánh giá chung và Video minh họa.....	58

DANH MỤC HÌNH ẢNH

Hình 2.1	Cảm biến US-100	16
Hình 2.2	Datasheet US-100	17
Hình 2.3	Mạch Raspberry Pico dùng chip RP2040	18
Hình 3.1	Electrical Clearance mặc định	27
Hình 3.2	Nguồn Electrical Clearance	27
Hình 3.3	Đường 3.3V cấp cho US100 + UART	28
Hình 3.4	Đường 3.3V cho RP2040	29
Hình 3.5	Đường 12V vào AMS1117	29
Hình 3.6	Đường tín hiệu UART 9600 baud	30
Hình 3.7	Lỗ khoan và Vòng đồng	31
Hình 3.8	Vùng Đồ Đồng Lốp Trên	32
Hình 3.9	Vùng Đồ Đồng Lốp Dưới	32
Hình 3.10	Bo mạch PCB	37
Hình 3.11, Hình 3.12, Hình 3.13, Hình 3.14, Hình 3.15	Lưu đồ PicoSort UART	39
Hình 3.16	Giao tiếp với giao diện	41
Hình 3.18	Hàm hỗ trợ CRC	42
Hình 3.17	Hàm setup	42
Hình 3.19, Hình 3.20	Hàm logic xử lý phím đọc dữ liệu và gửi 1000 gói tin	42
Hình 3.21	Hàm vòng lặp	43
Hình 3.22	Giao diện GUI	47
Hình 3.23	Sơ đồ tuần tự	48
Hình 4.1, Hình 4.2	Trạng thái nghỉ chân GPIO16 / Trạng thái gửi chân GPIO16	49
Hình 4.3	Chỉnh Time/div của OSC	50
Hình 4.4	Tín hiệu tương tự trên chân GPIO17	51
Hình 4.5	Que đo OSC tại chân GPIO 0	51
Hình 4.6	Que đo OSC tại chân GPIO 1	52
Hình 4.7	Đo Vin Ams1117 12V	53
Hình 4.8	Đo Vout Ams1117 5V	53
Hình 4.9	3V3 pico	53
Hình 4.10	bo mạch Pico khi đang ở chế độ READ, vật cản ở gần < 10cm	54
Hình 4.11	bo mạch Pico khi đang ở chế độ READ, vật cản ở xa > 10cm	54
Hình 4.12	màn hình giao diện C# sau khi hoàn tất bài TEST 1000 gói tin	55
Hình 4.13	Đo vật thể ở khoảng cách 10cm	56
Hình 4.14	Kết quả đo thực tế <10 cm và hiển thị trên giao diện	56
Hình 4.15	Kết quả đo thực tế 10cm và hiển thị trên giao diện	57
Hình 4.16	Kết quả đo thực tế 15cm và hiển thị trên giao diện	57

DANH MỤC TỪ VIẾT TẮT

(Xếp theo thứ tự A, B, C của chữ cái đầu viết tắt)

CRC	Cyclic Redundancy Check
GND	Ground
IC	Integrated Circuit
IOT	Internet Of Things
PCB	Printed Circuit Board
UART	Universal Asynchronous Receiver/Transmitter
VCC	Voltage Common Collector

MỞ ĐẦU

1. Đặt vấn đề

Bối cảnh của đề tài: Trong bối cảnh công nghệ nhúng và IoT đang phát triển mạnh mẽ, các cảm biến khoảng cách ngày càng được sử dụng rộng rãi và ứng dụng phổ biến cho nhiều thiết bị như: robot, xe tự hành, đo mực nước, các hệ thống cảnh báo... Việc tích hợp vi điều khiển và giao diện máy tính giúp nâng cao khả năng giám sát và điều khiển của người dùng.

Vì sao chọn đề tài: Đề tài có tính ứng dụng cao và dễ triển khai trong thời gian ngắn. Đề tài đơn giản, chi phí thấp phù hợp với học sinh sinh viên. Nó giúp người thực hiện rèn luyện kỹ năng về cả phần cứng và phần mềm.

Tính cấp thiết: Các hệ thống giám sát và đo lường khoảng cách là nền tảng cho nhiều ứng dụng thực tế như cảnh báo an toàn, quản lý mức chất lỏng, kiểm soát ra vào... Do đó, việc nghiên cứu và triển khai đề tài là cần thiết để phục vụ cho học tập, nghiên cứu và mở rộng các sản phẩm.

2. Mục tiêu của đề tài

- Xây dựng hệ thống đo khoảng cách bằng cảm biến siêu âm US-100.
- Lập trình vi điều khiển RP2040 để xử lý dữ liệu và truyền qua UART.
- Phát triển giao diện trên Visual C# để hiển thị số liệu đo được.
- Thiết kế giao diện trực quan, dễ sử dụng, có biểu diễn bằng số và biểu đồ.
- Đảm bảo hệ thống hoạt động ổn định, có khả năng mở rộng.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng: Cảm biến siêu âm US-100. Vi điều khiển RP2040 . Máy tính chạy ứng dụng Visual C#, CH340G. Người sử dụng hệ thống để quan sát, giám sát dữ liệu đo khoảng cách.

Phạm vi nghiên cứu:

- Thời gian thực hiện: từ ngày 10/9/2025 đến ngày 19/11/2025
- Nơi thực hiện: trong phòng lab hoặc nơi có vật thể cần đo khoảng cách.
- Kích thước và giới hạn: phạm vi đo từ 5cm đến 30 cm
- Đối tượng ảnh hưởng: người dùng trực tiếp và đối tượng được đo khoảng cách

4. Ý nghĩa thực tiễn của đề tài

Tính ứng dụng: hệ thống dùng để đo khoảng cách vật thể, đo mực chất lỏng hoặc điều khiển-giám sát robot vận hành.

Vấn đề giải quyết: hỗ trợ người dùng quan sát vật thể và thu thập dữ liệu một cách trực quan trên giao diện thay vì quan sát trên cảm biến.

Tín hiệu quả: tiết kiệm chi phí so với các hệ thống đo lường phức tạp, dễ mở rộng, nâng cao kỹ năng nhúng và lập trình giao diện.

CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI

1.1 Bối cảnh và lý do chọn đề tài

Trong bối cảnh khoa học – công nghệ phát triển mạnh mẽ, các hệ thống đo lường và giám sát tự động ngày càng được ứng dụng rộng rãi trong đời sống và sản xuất. Việc thu thập và xử lý dữ liệu một cách chính xác không chỉ giúp nâng cao hiệu quả quản lý mà còn góp phần đảm bảo an toàn và giảm thiểu sai sót trong quá trình vận hành. Chính vì vậy, nhiều mô hình và đề tài liên quan đến hệ thống đo và hiển thị thông số đã được các nhà nghiên cứu, sinh viên và kỹ sư thực hiện trong những năm gần đây.

Tuy nhiên, phần lớn các công trình trước chỉ dừng lại ở mức xây dựng mô hình cơ bản, chưa thật sự tối ưu về độ chính xác, tốc độ xử lý và tính tiện dụng. Một số đề tài chỉ mới tập trung vào phần đo lường mà chưa chú trọng đến tính ổn định của phần cứng; ngược lại, một số khác chỉ tập trung vào hiển thị mà chưa kết hợp chặt chẽ với thuật toán lọc và xử lý tín hiệu. Thêm vào đó, nhiều mô hình vẫn còn hạn chế ở khả năng mở rộng hoặc chưa tạo điều kiện thuận lợi cho người dùng thao tác.

Những hạn chế này mở ra nhu cầu cần có một hệ thống được thiết kế tích hợp – ổn định – chính xác – dễ sử dụng, từ đó nâng cao tính thực tiễn và khả năng triển khai. Đây chính là lý do nhóm chọn thực hiện đề tài này nhằm cải tiến và hoàn thiện những điểm còn thiếu sót của các nghiên cứu trước.

1.2 Tổng quan các nghiên cứu liên quan

Khi khảo sát một số công trình nghiên cứu trong và ngoài nước, có thể thấy rằng hướng nghiên cứu về các hệ thống đo lường điện tử đã được quan tâm từ khá lâu. Những nghiên cứu này thường tập trung vào ba nhóm chính:

1.2.1 Nhóm nghiên cứu về cảm biến và thu thập dữ liệu

Nhiều đề tài đã sử dụng các loại cảm biến để đo các đại lượng như điện áp, dòng điện, khoảng cách, nhiệt độ,... Tuy nhiên độ nhạy, độ ổn định và khả năng chống nhiễu của cảm biến vẫn là vấn đề cần tiếp tục cải thiện. Một số nghiên cứu chỉ sử dụng

phương pháp đo trực tiếp mà chưa áp dụng các kỹ thuật lọc tín hiệu, dẫn đến kết quả đo còn dao động và thiếu nhất quán.

1.2.2 Nhóm nghiên cứu về xử lý tín hiệu và thuật toán

Một số bài báo đã đề cập đến việc sử dụng các thuật toán lọc số như lọc trung bình, trung bình trượt hoặc Kalman nhằm giảm nhiễu trong quá trình đo. Tuy nhiên, các hệ thống mẫu chỉ mới áp dụng ở mức thử nghiệm, chưa kết hợp đồng bộ giữa phần mềm và phần cứng để mang lại hiệu quả tối ưu.

1.2.3 Nhóm nghiên cứu về giao diện người dùng và hiển thị

Các đề tài dùng LCD 16x2, LCD 20x4, màn hình OLED hoặc giao diện máy tính để hiển thị kết quả đo. Tuy nhiên nhiều mô hình vẫn còn hạn chế về tính trực quan, thiếu khả năng cập nhật linh hoạt, hoặc chưa hỗ trợ hiển thị nhiều thông số trong cùng một thời điểm.

Từ việc tổng hợp các nghiên cứu trên, có thể rút ra một số kinh nghiệm quan trọng:

- Lựa chọn cảm biến phù hợp là yếu tố quyết định độ chính xác của hệ đo.
- Việc kết hợp giữa thuật toán lọc và xử lý tín hiệu là cần thiết để đảm bảo dữ liệu ổn định.
- Thiết kế phần cứng phải gọn, an toàn và có khả năng hoạt động lâu dài.
- Giao diện hiển thị cần đơn giản, rõ ràng và dễ quan sát.

Những kinh nghiệm này đóng vai trò làm nền tảng cho quá trình xây dựng và triển khai đề tài.

1.3 Những hạn chế hiện tại và vấn đề đề tài cần giải quyết

Mặc dù đã có nhiều nghiên cứu trước đó, nhưng vẫn tồn tại những vấn đề chưa được giải quyết triệt để:

- Hạn chế về độ chính xác: Nhiều hệ thống chưa có cơ chế lọc hoặc hiệu chỉnh dữ liệu nên kết quả đo dao động và thiếu ổn định. Điều này ảnh hưởng đến khả năng ứng dụng thực tế.

- Hạn chế về thiết kế phần cứng: Một số mô hình sử dụng phần cứng rời rạc, dây nối phức tạp, dễ gây nhiễu hoặc mất tín hiệu. Ngoài ra, khả năng mở rộng cho các tính năng mới còn thấp.
- Hạn chế về giao diện người dùng: Việc hiển thị còn đơn giản, chưa trực quan và chưa hỗ trợ nhiều chế độ quan sát. Một số hệ thống không cung cấp thông tin đầy đủ, gây khó khăn cho người sử dụng.
- Hạn chế về tích hợp: Đa phần các hệ thống chỉ chú trọng vào một thành phần (đo, xử lý hoặc hiển thị) mà chưa kết hợp tất cả thành một giải pháp hoàn chỉnh.

1.4 Mục tiêu cải tiến của đề tài

Đề tài được xây dựng với định hướng khắc phục những hạn chế nêu trên thông qua các giải pháp:

- Thiết kế hệ thống phần cứng tối ưu, ổn định và dễ mở rộng.
- Áp dụng thuật toán xử lý nhằm tăng độ chính xác và giảm nhiễu tín hiệu.
- Tích hợp đầy đủ chức năng đo – xử lý – hiển thị trong một mô hình thống nhất.
- Xây dựng giao diện hiển thị rõ ràng, trực quan và dễ tương tác.
- Đảm bảo hệ thống hoạt động ổn định trong thời gian dài.

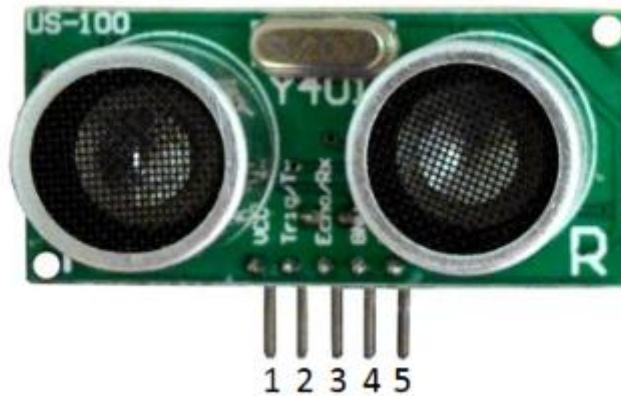
Với những mục tiêu này, đề tài hướng tới việc tạo ra một mô hình hoàn chỉnh, khả thi và có tính ứng dụng cao, góp phần nâng cao chất lượng trong quá trình nghiên cứu và thực hành.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Nguyên lý hoạt động của linh kiện

2.1.1 Cảm biến siêu âm US-100

Cảm biến siêu âm US-100 là loại cảm biến đo khoảng cách bằng sóng siêu âm có độ chính xác cao. Thiết bị hoạt động dựa trên nguyên lý phát và thu sóng âm trong không khí, sau đó tính toán thời gian phản hồi để xác định khoảng cách đến vật thể.



Hình 2.1 Cảm biến US-100

Nguyên lý hoạt động:

1. Cảm biến phát ra một chùm sóng siêu âm tần số khoảng 40 kHz từ bộ phát (Trigger).
2. Sóng âm truyền đi trong không khí và gặp vật cản sẽ phản xạ trở lại.
3. Bộ thu (Echo) nhận sóng phản xạ và gửi tín hiệu về vi điều khiển.
4. Vi điều khiển tính toán thời gian sóng đi và về (Δt), từ đó tính khoảng cách

Theo công thức:

$$S = \frac{v \times \Delta t}{2} \quad (2-1)$$

Trong đó:

- S : Khoảng cách (cm)
- v : Vận tốc âm thanh trong không khí (~ 340 m/s)
- Δt : Thời gian sóng đi và về (giây)

Thông số kỹ thuật:

- Input voltage: 5V DC
- Quiescent current: less than 2mA
- Level output: 5V high
- Level output: at the end of 0V
- Induction angle: not more than 15 degrees
- Detection distance: 2cm-450cm
- Precision: up to 1mm
- Dimensions: 4.4cm x 2.6cm x 1.4cm
- Weight: 43g

Hình 2.2 Datasheet US-100

Đặc điểm nổi bật so với HC-SR04:

- Có thể giao tiếp UART trực tiếp, giảm sai lệch do đo thời gian.
- Có mạch xử lý tín hiệu tích hợp, giảm nhiễu.
- Hoạt động ổn định hơn trong môi trường có nhiễu điện.

2.1.2 Vi điều khiển RP2040

RP2040 là vi điều khiển 32-bit do Raspberry Pi Foundation phát triển, được ứng dụng rộng rãi trong các thiết bị nhúng.



Hình 2.3 Mạch Raspberry Pico dùng chip RP2040

Cấu trúc phần cứng chính:

- CPU: Dual-core ARM Cortex-M0+ @ 133 MHz
- RAM: 264 KB
- Bộ nhớ Flash: 2 MB
- 30 chân GPIO hỗ trợ UART, SPI, I2C, PWM, ADC
- Điện áp hoạt động: 3.3V

Chức năng trong đề tài:

- Nhận dữ liệu đo từ cảm biến US-100 qua UART.
- Xử lý, chuyển đổi dữ liệu sang đơn vị cm.
- Gửi kết quả đo sang máy tính qua cổng UART – USB (CH340G).

Ưu điểm:

- Xử lý nhanh, độ trễ thấp.
- Hỗ trợ nhiều giao tiếp ngoại vi.
- Giá thành rẻ, dễ lập trình với ngôn ngữ C/C++ hoặc MicroPython.

2.1.3 Opto cách ly

Optocoupler là linh kiện điện tử có chức năng cách ly quang học giữa hai mạch điện riêng biệt — thường là mạch công suất và mạch điều khiển — nhằm bảo vệ vi điều khiển khỏi nhiễu hoặc điện áp cao.

Cấu tạo: Opto gồm một LED phát sáng và một phototransistor (hoặc photodiode) nằm trong cùng một vỏ kín. Khi dòng điện chạy qua LED, ánh sáng phát ra kích hoạt transistor, cho phép dòng điện đi qua phía đầu ra mà không cần nối điện trực tiếp.

Nguyên lý hoạt động:

- Khi vi điều khiển xuất tín hiệu điều khiển, dòng điện nhỏ đi qua LED trong opto.
- LED phát sáng, kích hoạt phototransistor ở đầu ra.
- Nhờ đó, tín hiệu được truyền qua mà vẫn giữ cách ly điện giữa hai mạch.

Ứng dụng trong đề tài:

- Bảo vệ vi điều khiển RP2040 khỏi nhiễu điện áp từ cảm biến hoặc thiết bị ngoại vi.
- Giúp tín hiệu UART hoặc điều khiển ổn định và an toàn hơn.
- Có thể dùng để kích hoạt cảnh báo (LED hoặc còi) mà không ảnh hưởng đến mạch chính.

Ưu điểm:

- Cách ly điện tuyệt đối giữa hai mạch.
- Giảm nhiễu xung, tăng độ tin cậy.
- Dễ tích hợp, giá thành rẻ.

Một số loại opto phổ biến: PC817, 4N25, TLP521, 6N137...

Features

- Very high speed-10 MBit/s
- Superior CMR-10 kV/μs
- Double working voltage-480V
- Fan-out of 8 over -40°C to +85°C
- Logic gate output
- Strobable output
- Wired OR-open collector
- U.L. recognized (File # E90700)

Hình 2.4 Datasheet 6N137

...Để LED trong opto hoạt động, cần một điện trở hạn dòng. Giá trị điện trở này được tính theo Định luật Ohm, đảm bảo dòng điện thuận (I_F) nằm trong mức cho phép của nhà sản xuất (thường từ 5mA - 15mA).

$$R = \frac{v_{source} - v_F}{I_F} \quad (2-2)$$

Trong đó:

- **R**: Giá trị điện trở hạn dòng (Ohm).
- **V_{source}**: Điện áp nguồn cấp cho LED (Volt).
- **V_F**: Điện áp rơi thuận (Forward Voltage) của LED bên trong opto (Volt).
- **I_F**: Dòng điện thuận (Forward Current) mong muốn (Ampere).

2.2 Nguyên lý giao thức truyền nhận dữ liệu UART

UART (Universal Asynchronous Receiver/Transmitter) là giao thức truyền thông nối tiếp không đồng bộ, được sử dụng phổ biến trong truyền dữ liệu giữa vi điều khiển và máy tính.

2.2.1 Cấu trúc truyền UART

Dữ liệu được truyền theo khung (frame) gồm:

Start bit | 8 Data bits | Optional Parity | 1–2 Stop bits

Khi truyền, cả hai thiết bị phải cùng tốc độ baud rate (ví dụ 9600 bps).

2.2.2 Nguyên lý hoạt động

- Bên phát: Chuyển dữ liệu song song (trong bộ nhớ) thành chuỗi bit nối tiếp, gửi qua chân TX.
- Bên thu: Nhận dữ liệu nối tiếp qua chân RX, giải mã và chuyển lại thành dữ liệu song song.

2.2.3 Ứng dụng trong đề tài

- RP2040 sử dụng UART để gửi dữ liệu đo khoảng cách tới máy tính.
- Máy tính sử dụng Visual C# để đọc dữ liệu qua cổng COM.
- Giao tiếp thông qua chip chuyển đổi USB-UART CH340G.

2.2.4 Kỹ thuật Software UART (Bit-Banging)

Software UART, hay còn gọi là "Bit-Banging", là một kỹ thuật lập trình cho phép mô phỏng chức năng của một bộ UART phần cứng bằng cách sử dụng các chân GPIO tiêu chuẩn và điều khiển thời gian chính xác bằng phần mềm.

Nguyên lý: Kỹ thuật này dựa trên việc phần mềm (firmware) tự tạo ra hoặc tự đọc các bit dữ liệu (Start, Data, Stop) theo đúng khung truyền (frame) của giao thức UART, với thời gian của mỗi bit được tính toán chính xác.

Công thức cốt lõi để xác định thời gian cho mỗi bit (Bit Period) là:

với tốc độ 9600 Baud:

$$\text{BitPeriod}(\mu\text{s}) = \frac{1,000,000}{\text{BaudRate}} = \frac{1,000,000}{9600} = 104.17\mu\text{s} \quad (2-3)$$

- **Khi truyền (TX):** Vì điều khiển sẽ cấu hình chân GPIO là OUTPUT và chủ động kéo mức tín hiệu (Start bit = LOW, Stop bit = HIGH) và giữ mỗi trạng thái trong đúng 104.17 μs .
- **Khi nhận (RX):** Vì điều khiển cấu hình chân là INPUT, phát hiện cạnh xuống (Start bit), sau đó chờ một khoảng thời gian (ví dụ: 1.5 * BitPeriod) để dịch

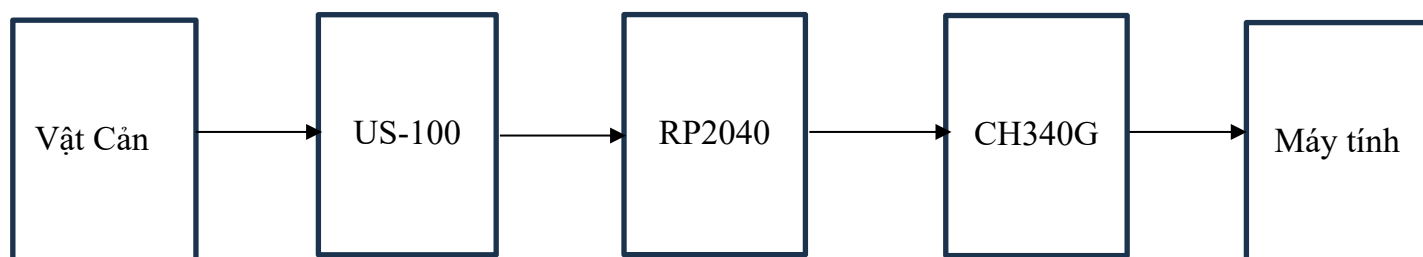
chuyển đến giữa bit dữ liệu đầu tiên, và tiếp tục lấy mẫu tại mỗi khoảng 1 * BitPeriod.

2.3 Nguyên lý mạch đo khoảng cách

Mạch đo được thiết kế gồm ba phần chính:

1. Phần phát – thu siêu âm (US-100): Tạo và nhận sóng siêu âm, chuyển đổi tín hiệu phản xạ thành tín hiệu điện.
2. Phần xử lý (RP2040): Đo thời gian phản hồi, tính toán khoảng cách theo công thức vật lý.
3. Phần truyền dữ liệu: Gửi dữ liệu khoảng cách đã xử lý sang máy tính qua UART – USB.

Sơ đồ khối hoạt động:



2.4 Thuật toán và phương pháp xử lý tín hiệu

2.4.1 Thuật toán đo khoảng cách

1. Gửi lệnh kích hoạt cảm biến.
2. Đợi tín hiệu phản hồi từ Echo hoặc dữ liệu UART.
3. Tính thời gian phản xạ (Δt).
4. Áp dụng công thức $S = v \times \Delta t / 2$.
5. Gửi kết quả sang máy tính qua UART.

2.4.2 Phương pháp xử lý nhiễu

Lấy trung bình cộng nhiều mẫu đo liên tiếp để giảm sai số:

$$S_{tb} = \frac{S_1 + S_2 + \dots + S_n}{n} \quad (2-4)$$

Bỏ qua các giá trị sai lệch lớn (nhiều, mất tín hiệu).

Giới hạn vùng đo hợp lệ (5 – 30 cm).

2.4.3 Phương pháp hiển thị dữ liệu

Máy tính nhận dữ liệu qua cổng COM.

Dữ liệu hiển thị theo 2 dạng:

- Dạng số: Hiển thị giá trị đo tức thời.
- Dạng biểu đồ: Biểu diễn sự thay đổi khoảng cách theo thời gian.

2.5 Thuật toán CRC

2.5.1 Khái niệm

CRC (Cyclic Redundancy Check) là một kỹ thuật kiểm tra lỗi phổ biến trong truyền thông nối tiếp. CRC giúp phát hiện các lỗi phát sinh trong quá trình truyền dữ liệu giữa hai thiết bị, từ đó đảm bảo gói tin nhận về là chính xác.

Trong đề tài này, để giảm độ phức tạp và tối ưu cho vi điều khiển, hệ thống sử dụng CRC dạng tổng kiểm tra (Checksum CRC-8) với giá trị nằm trong khoảng 0–255 (1 byte).

2.5.2 Công thức CRC sử dụng trong đề tài

Thuật toán CRC được định nghĩa như sau:

Công thức:

$$CRC = (\sum_{i=1}^n data[i]) \bmod 256 \quad (2-5)$$

Trong đó:

- $data[i]$: là giá trị byte của từng ký tự trong chuỗi dữ liệu.
- n : là số ký tự của chuỗi.
- Modulo 256: đảm bảo kết quả CRC nằm trong 1 byte (0–255).

2.5.3 Thuật toán CRC (diễn giải)

Khởi tạo biến tổng sum = 0

Với mỗi ký tự trong chuỗi dữ liệu:

- Chuyển ký tự sang mã ASCII (ví dụ: 'A' → 65)
- Cộng vào biến sum
- Sau khi cộng hết:
- Lấy sum % 256 để giới hạn trong 1 byte
- Gắn CRC này vào cuối gói tin để gửi đi

2.6 Các nội dung lý thuyết liên quan khác

2.6.1 Giao diện Visual C#

Ngôn ngữ C# được sử dụng để:

- Mở kết nối COM.
- Đọc dữ liệu từ vi điều khiển.
- Hiển thị giá trị trên giao diện và biểu đồ.

Các thư viện chính: System.IO.Ports, System.Windows.Forms, System.Drawing.

2.6.2 Chip chuyển đổi CH340G

- Chuyển tín hiệu UART (TX/RX) sang USB để máy tính nhận dạng.
- Hỗ trợ tốc độ truyền lên đến 1 Mbps, tương thích tốt với Windows.

2.6.3 Tiêu chuẩn thiết kế PCB (IPC-2221)

IPC-2221 là một tiêu chuẩn công nghiệp phổ biến cho thiết kế bo mạch in. Một trong những tính toán quan trọng là xác định độ rộng tối thiểu của đường mạch (Trace Width) để đảm bảo khả năng mang dòng điện an toàn mà không bị quá nhiệt.

Công thức tính toán (cho lớp ngoài) là:

$$W = \frac{I}{k \times (\Delta T)^{0.44}} \quad (2-6)$$

Trong đó:

- **W:** Độ rộng đường mạch (mm).
- **I:** Dòng điện tối đa (Ampere).
- **k:** Hằng số (thường là 0.048 cho lớp ngoài).
- **ΔT :** Độ tăng nhiệt độ cho phép (ví dụ: 10°C).

2.6.4 Nguyên lý Ổn áp Zener (*Zener Regulator*)

Diode Zener là một loại diode đặc biệt, khi được phân cực ngược, nó sẽ duy trì một điện áp gần như không đổi (V_Z) trên hai đầu của nó.

Tính chất này được ứng dụng để tạo ra các mạch ổn áp đơn giản, bao gồm một điện trở hạn dòng (R_s) mắc nối tiếp và Diode Zener ($Z1$) mắc song song với tải (R_L). Công thức tính điện trở R_s (tương ứng với $R1$ trong thiết kế) là:

$$R_s = \frac{V_{in} - V_Z}{I_Z + I_L} \quad (2-7)$$

Trong đó:

- **V_{in} :** Điện áp đầu vào (ví dụ: 5V từ USB).
- **V_Z :** Điện áp Zener mong muốn (ví dụ: 3.3V).
- **I_Z :** Dòng điện tối thiểu để Zener hoạt động ổn định.
- **I_L :** Dòng điện tiêu thụ tối đa của tải.

CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Tính toán các giá trị cho schematic

Việc lựa chọn và tính toán linh kiện được dựa trên mục tiêu thiết kế một bo mạch giao tiếp Raspberry Pi Pico với máy tính, có cách ly quang hoàn toàn.

Khối Nguồn Vi Điều Khiển (Pico):

- Sử dụng IC ổn áp AMS1117-5.0 (U1) để tạo ra nguồn 5V ổn định, cấp cho chân VBUS (Pin 40) của Pico .
- Tụ lọc đầu vào: C1 (220uF) (lọc chính) và C2 (0.1uF) (lọc nhiễu cao tần).
- Tụ lọc đầu ra: C4 (22uF) (đảm bảo ổn định LDO) và C3 (0.1uF) (lọc nhiễu cao tần). Khối Nguồn Giao Tiếp (USB):
- Sử dụng mạch ổn áp Zener gồm điện trở R1 (220 Ohm) và Diode Zener (Z1) để tạo nguồn 3.3V từ 5V của cổng USB . Nguồn 5V cấp cho chip CH340G và mạch kéo lên của Opto U5 .

Khối Chuyển Đổi USB-UART: Sử dụng IC CH340G (U4) . IC này yêu cầu thạch anh Y1 (12MHz) và hai tụ C5, C6 (22pF) để hoạt động, đây là các giá trị tiêu chuẩn theo datasheet của nhà sản xuất.

Khối Cách Ly Quang: Sử dụng 2 IC 6N137 (U5, U6) . Đây là IC cách ly quang tốc độ cao (lên đến 10Mbit/s), được chọn để đảm bảo tín hiệu UART (thường chạy ở 9600 - 115200 bps) không bị méo hoặc suy hao.

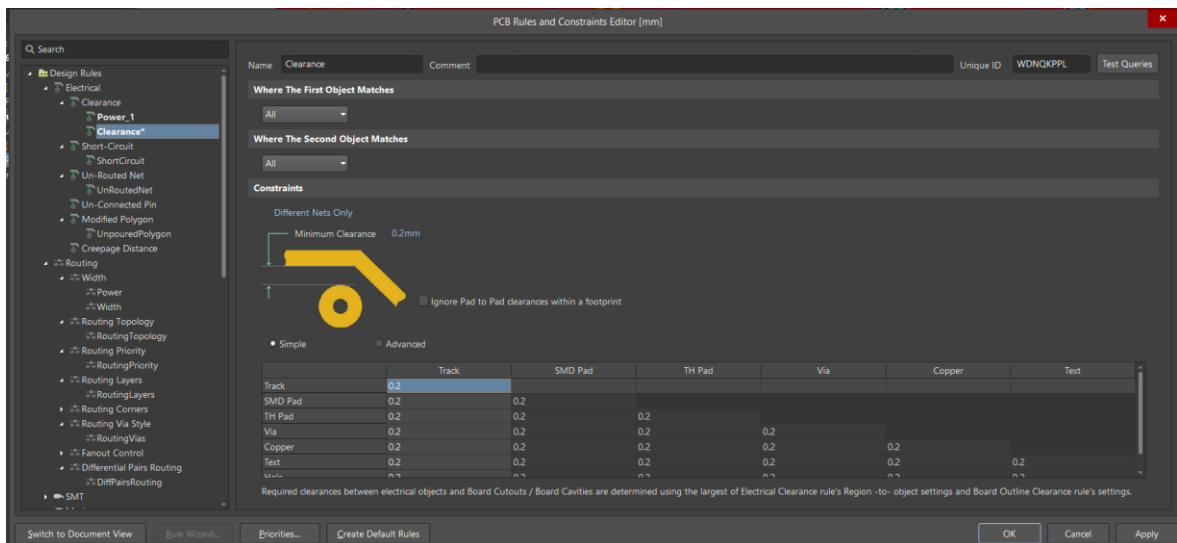
3.2 Tính toán cho PCB

3.2.1 Khoảng cách (Clearance) trong thiết kế PCB

3.2.1.1 Electrical Clearance (khoảng cách điện)

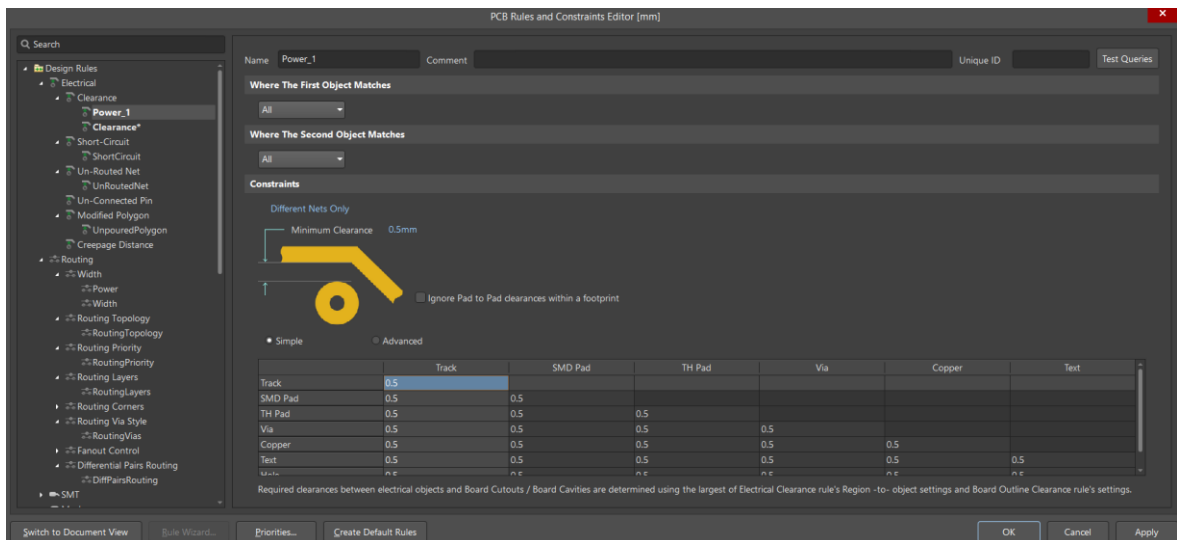
Dựa vào mức điện áp làm việc của hệ thống:

Khối logic RP2040 – UART – US100 (3.3V/5V): chỉ cần clearance 0.2–0.5 mm, vì điện áp thấp và ít nguy cơ đánh thủng.



Hình 3.1 Electrical Clearance mặc định

Khối nguồn 12V → AMS1117-5V/3V3: cần clearance lớn hơn, tối thiểu 0.5 mm để đảm bảo không phóng điện và giảm nhiễu.



Hình 3.2 Nguồn Electrical Clearance

3.2.1.2 Độ rộng đường mạch (Line Width)

Áp dụng công thức theo chuẩn IPC-2221 (Outer Layer):

$$W = \frac{I}{k \cdot (\Delta T)^{0.44}} \frac{1}{0.725} \quad (3-1)$$

Dựa vào dòng tiêu thụ thực tế:

3.2.1.3 Đường 3.3V cấp cho US100 + UART (<200 mA)

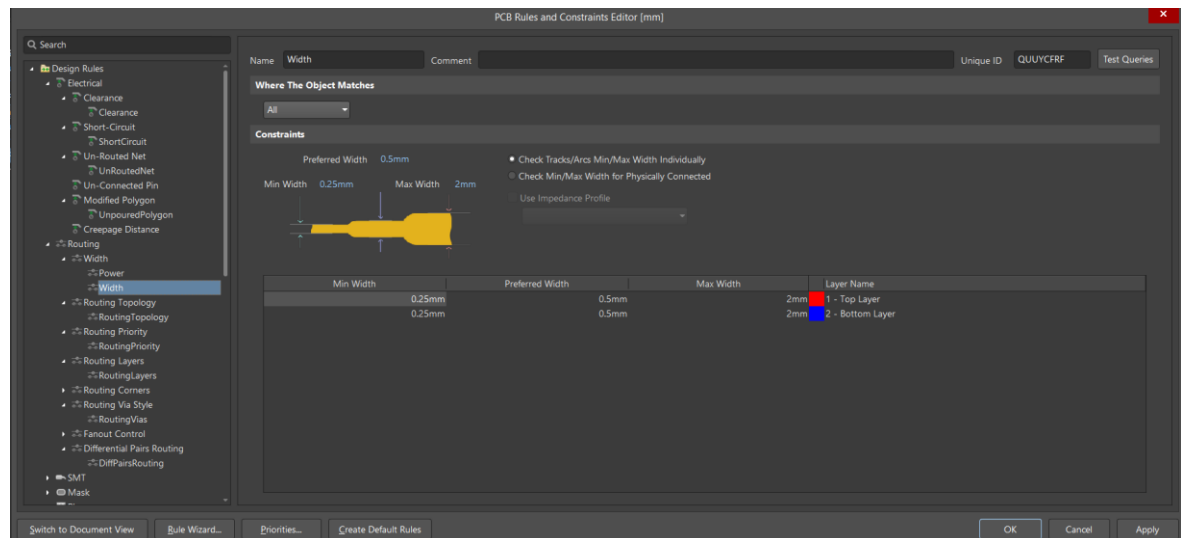
Dòng tải tối đa: 0.2 A

Tính toán theo IPC-2221:

$$W = \frac{0.2}{0.048 \cdot (10)^{0.44}} \frac{1}{0.725} \quad (3-2)$$

$W \approx 0.25\text{--}0.28 \text{ mm}$

Kết luận: Chọn Min Width (Tối thiểu): 0.25mm.



Hình 3.3 Đường 3.3V cấp cho US100 + UART

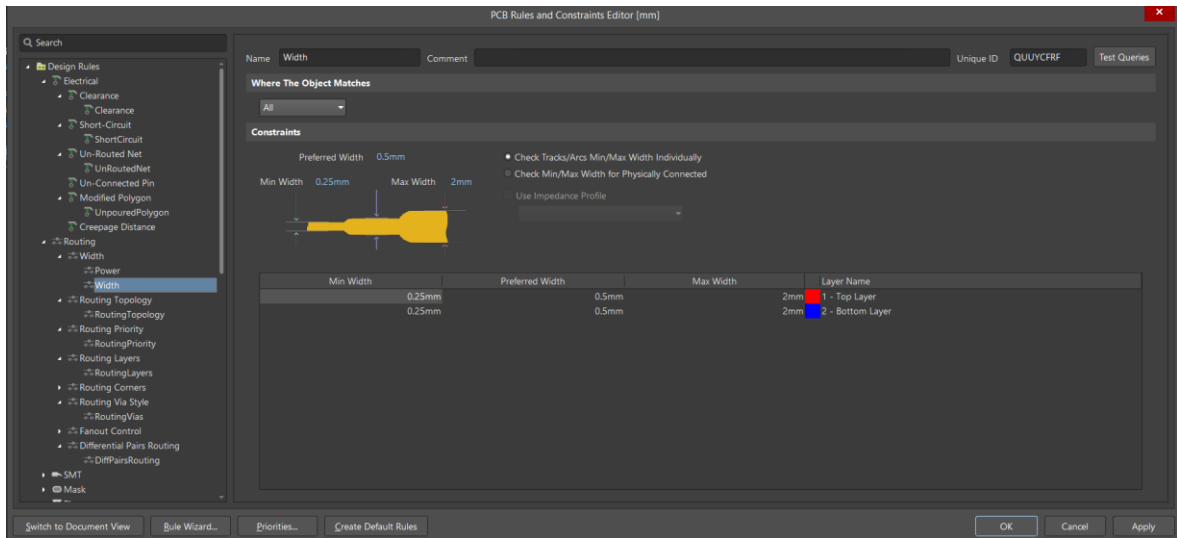
3.2.1.4 Đường 3.3V cho RP2040 (120–150 mA)

Dòng tối đa: 0.15 A

$$W = \frac{0.15}{0.048 \cdot (10)^{0.44}} \frac{1}{0.725} \quad (3-3)$$

$W \approx 0.22 \text{ mm}$

Kết luận: Chọn Width (Mặc định): 0.5mm.



Hình 3.4 Đường 3.3V cho RP2040

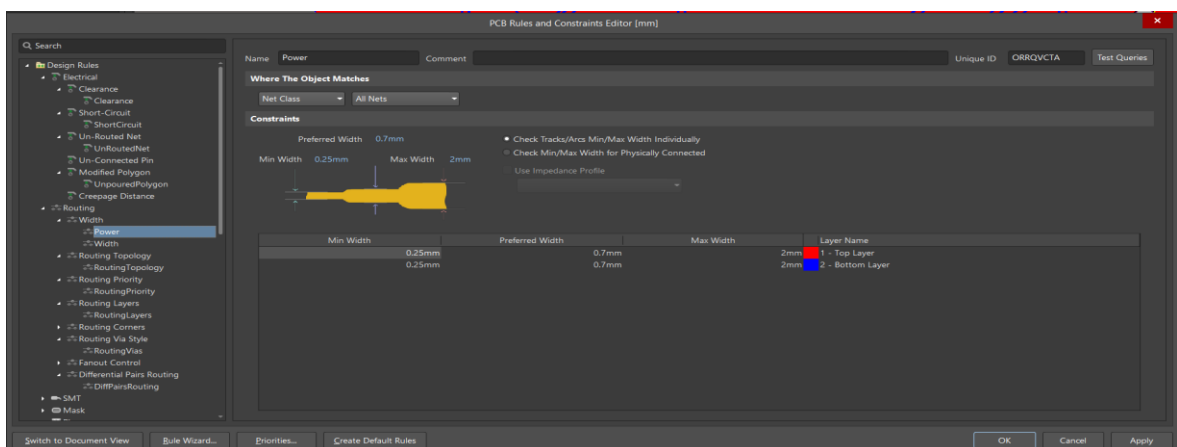
3.2.1.5 Đường 12V vào AMS1117 (1-2A)

Dòng tối đa: 1A (AMS1117 chịu dòng tối đa 1A)

$$W = \frac{1}{0.048 \cdot (10)^{0.44}}^{1/0.725} \quad (3-4)$$

$W \approx 0.41402 \text{ mm}$

Kết luận: Chọn Preferred Width ('Power'): 0.7mm. Mạch nguồn công suất thấp – giúp giảm sụt áp và cải thiện tản nhiệt đường mạch.

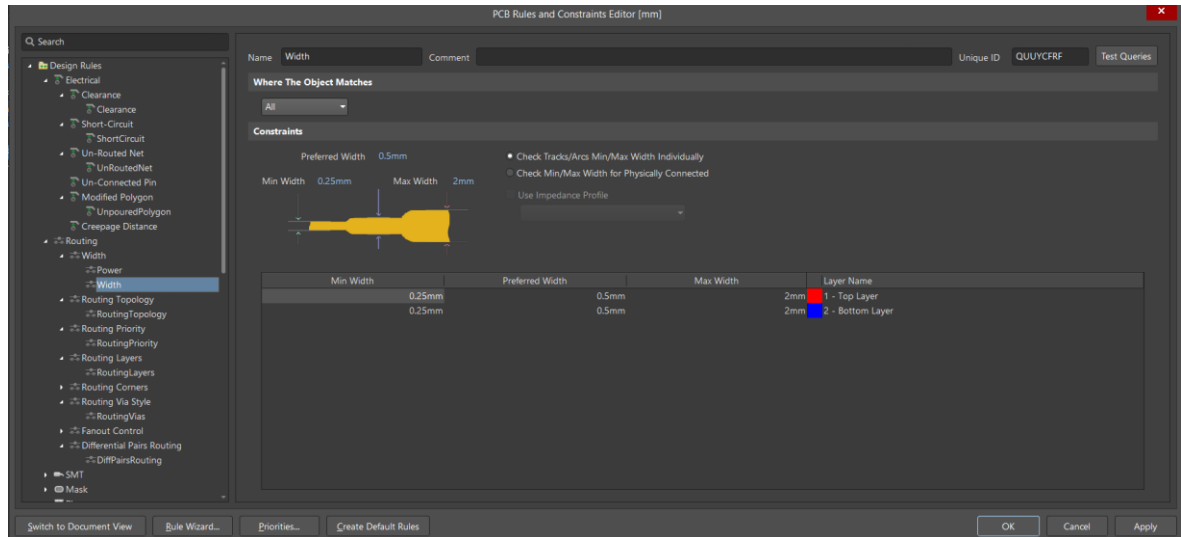


Hình 3.5 Đường 12V vào AMS1117

3.2.1.6 Đường tín hiệu UART 9600 baud (TX/RX)

Tốc độ 9600 baud → tín hiệu thấp tần, dòng gần như 0 A.

Kết luận: Trace width 0.25 mm là tối ưu để đi dây và đủ trở kháng cho tín hiệu TTL 3.3V.



Hình 3.6 Đường tín hiệu UART 9600 baud

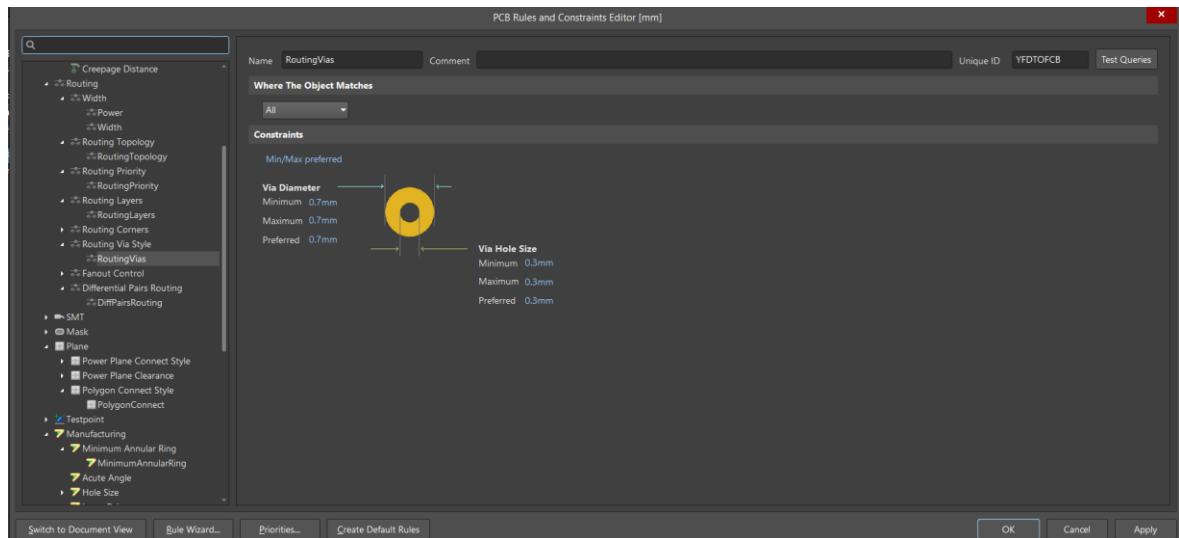
3.2.2 Quy tắc Lỗ khoan và Vòng đồng (Routing Vias & Annular Ring)

Giá trị Cấu hình:

- Routing Vias: Lỗ (Hole) 0.3mm / Đường kính (Diameter) 0.7mm.
- Minimum Annular Ring: Tối thiểu 0.2mm (Theo nhà sản xuất gia công).

Nguyên lý:

- Quy tắc Via xác định kích thước lỗ khoan và pad, trong khi Annular Ring xác định độ rộng tối thiểu của vòng đồng xung quanh lỗ để đảm bảo kết nối.
- Với Via Diameter là 0.7mm và Via Hole Size là 0.3mm, độ rộng vòng đồng (annular ring) thực tế được tạo ra là: $(0.7\text{mm} - 0.3\text{mm}) / 2 = 0.2\text{mm}$



Hình 3.7 Lỗ khoan và Vòng đồng

3.2.3 Polygon Pour (Vùng Đổ Đồng)

Polygon Pour (Vùng Đổ Đồng) là một kỹ thuật thiết kế PCB tiên tiến, trong đó một vùng lớn của bo mạch được phủ đồng và kết nối với một net cụ thể (thường là GND hoặc VCC).

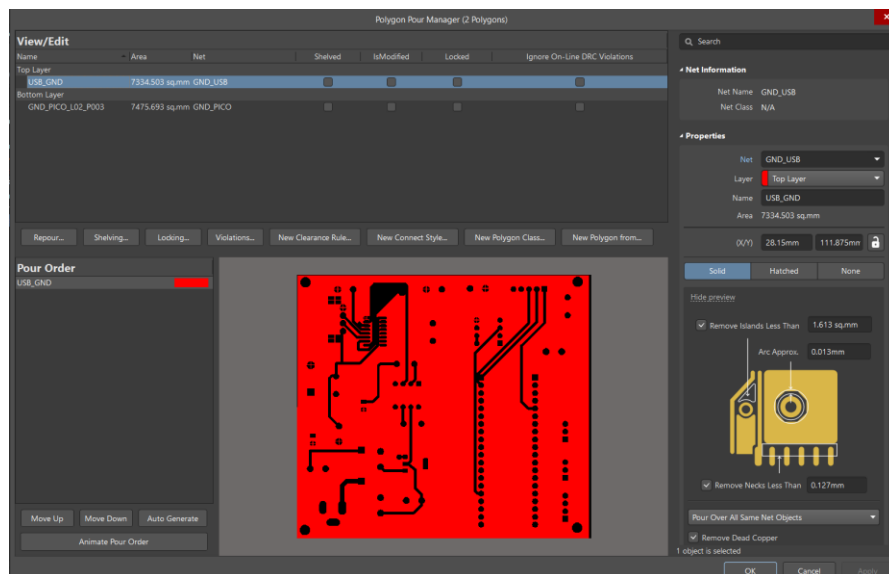
3.2.3.1 Cấu hình Polygon Pour

Để đảm bảo tính toàn vẹn tín hiệu và an toàn cách ly điện:

Vùng Đổ Đồng Lớp Trên (Top Layer):

- Tên (Name): USB_GND
- Lớp (Layer): Top Layer
- Kết nối Net (Net): GND_USB
- Diện tích (Area): 7334.503 sq.mm

Nguyên lý: Vùng đồng này tạo ra một mặt phẳng GND (Ground USB) cho toàn bộ Miền 2 (Miền An toàn), kết nối với cổng USB của máy tính.

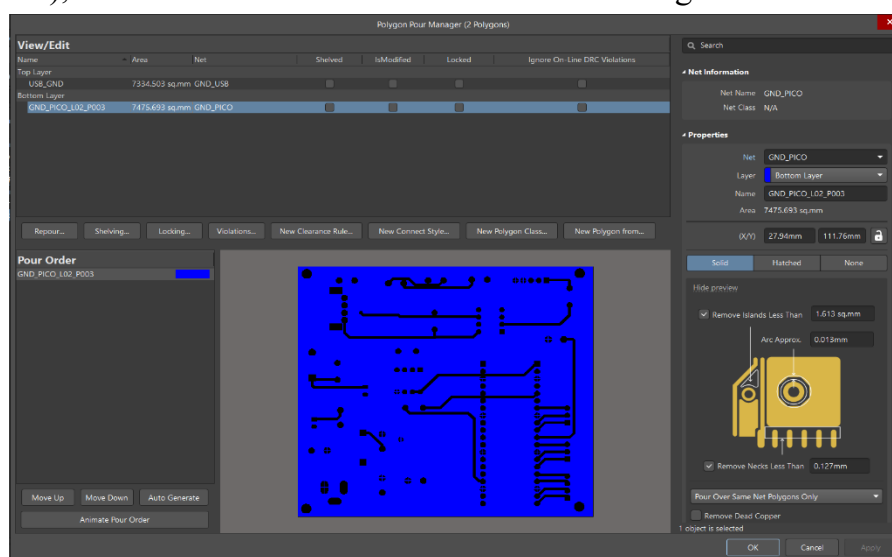


Hình 3.8 Vùng Đồ Đồng Lớp Trên

Vùng Đồ Đồng Lớp Dưới (Bottom Layer):

- Tên (Name): GND_PICO_L02_P003
- Lớp (Layer): Bottom Layer
- Kết nối Net (Net): GND_PICO
- Diện tích (Area): 7475.693 sq.mm

Nguyên lý: Vùng đồng này tạo ra một mặt phẳng GND riêng biệt cho toàn bộ Miền 1 (Miền 5V), kết nối với vi điều khiển RP2040 và khối nguồn.



Hình 3.9 Vùng Đồ Đồng Lớp Dưới

3.2.3.2 Kết luận kỹ thuật

Hai Polygon Pour được bố trí ở hai lớp khác nhau và kết nối với hai net độc lập (GND_USB và GND_PICO), đảm bảo phân tách hai miền mass theo đúng nguyên tắc cách ly tín hiệu. Cấu hình này hỗ trợ giảm nhiễu, ổn định nguồn và phục vụ đúng thiết kế của mạch có sử dụng opto cách ly.

3.2.3.3 Region

- Dùng để vẽ hình dạng polygon tự do.
- Kết nối trực tiếp vào net (GND, 3V3, 5V).
- Dùng để tạo Polygon Cutout giúp tách riêng vùng nguồn, tránh nhiễu giữa 12V và tín hiệu UART/RP2040.
- Phù hợp để tùy chỉnh pad đặc biệt và bo cạnh PCB.

3.3 Tính toán các giá trị theo các công thức lý thuyết

3.3.1 Tính toán Khối Nguồn (U1: AMS1117-5.0)

Chuyển đổi 12V đầu vào xuống 5V (VBUS) cho Pi Pico.

Tính toán Công suất Tiêu tán (PD):

Bộ ổn áp tuyến tính (LDO).

- Điện áp vào U1 (V_{in}): Nguồn 12V (từ J2) đi qua Diode Schottky D2 (1N5819). Diode này có sụt áp thuận (V_f) thấp, khoảng 0.4V.

$$V_{in} = V_{jack} - V_f = 12V - 0.4V = 11.6V \quad (3-5)$$

- Điện áp ra U1 (V_{out}): Cố định 5V.
- Dòng tải (I_L): dòng tải thiết kế (Pico + Cảm biến + Opto + Hệ số an toàn) được ước tính là 0.33A.

$$P_D = V_{in} - V_{out} \times I_o = 11.6V - 5V = 6.7V \quad (3-6)$$

3.3.2 Mạch Zener ($R1 = 220 \Omega \rightarrow Z1 = 3.3V$) — 3V3_LAP

Tính dòng qua R1 ($V_{in} = 5.0V$ giả định):

Mạch sử dụng nguồn vào 5V (V_{in}) và diode Zener 3.3V (V_{out}) để tạo mức điện áp tham chiếu 3.3V_LAP.

Điện áp rơi trên điện trở R1:

$$V_{R1} = V_{in} - V_{out} = 5V - 3.3V = 1.7V \quad (3-7)$$

Dòng điện qua R1:

$$I_{R1} = \frac{1.7V}{220\Omega} = 7.73mA \quad (3-8)$$

Net 3.3V_LAP không tải:

$$I_L = 0A \Rightarrow I_Z = I_{R1} = 7.73mA \quad (3-9)$$

3.3.3 CH340G (USB↔UART)

3.3.3.1 Điện áp & dòng

Cấp bằng 5 V (V_{BUS}), nhưng CH340G có chân VCC (theo schematic bạn nối VCC vào 3V hoặc 5V tùy cấu hình) — thường dùng 5 V với nội bộ có LDO; nhiều module dùng 3.3V.

Dòng tiêu thụ hoạt động tầm 20–50 mA + phụ thuộc traffic USB.

3.3.3.2 Tụ và thạch anh

Crystal: 12 MHz, CL (datasheet crystal) $\approx$ 20 pF.

Theo công thức:

$$C_L = \frac{C_1 \times C_2}{C_1 + C_2} + C_{stray} = \frac{22pF \times 22pF}{22pF + 22pF} + 4pF = 11pF + 4pF = 15pF \quad (3-10)$$

Trong đó:

- Các tụ này (C5, C6) là tụ tải (load capacitors). Giá trị của chúng phải phù hợp với C_L (Load Capacitance) của thạch anh ($\sim$ 12pF-20pF).
- C_{stray} là điện dung ký sinh của bo mạch ($\sim$ 2-5pF), giả định 4pF.

3.3.4 Phân tích và Tính toán Khối Cách ly Tín hiệu (U5, U6 – Optocoupler 6N137)

3.3.4.1 Thông số yêu cầu theo lý thuyết

Điện áp LED bên trong 6N137: $V_f = 1.4V$ (3-11)

Dòng kích hoạt tối thiểu: $I_F \geq 5mA$ (3-12)

Dòng khuyến nghị: $I_F = 7.5 - 10mA$ (3-13)

3.3.4.2 Tính toán dòng LED cho U6 (Pico TX $\rightarrow$ USB RX)

Công thức:

$$I_F = \frac{V_{in} - V_f}{R_3} = \frac{3.3V - 1.4V}{200\Omega} = 9.5mA \quad (3-14)$$

Trong đó:

- V_{in} : điện áp từ Pico
- V_f : sụt áp LED 6N137
- R_3 : điện trở hạn dòng

Kết luận:

Dòng 9.5 mA phù hợp, nằm trong vùng hoạt động tối ưu của 6N137.

3.3.4.3 Tính toán dòng LED cho U5 (PC TX $\rightarrow$ Pico RX)

Công thức:

$$I_F = \frac{V_{in} - V_f}{R_5} = \frac{3.3V - 1.4V}{200\Omega} = 9.5mA \quad (3-15)$$

Trong đó:

- V_{in} : nguồn từ mạch Zener
- V_f : sụt áp LED 6N137
- R_5 : điện trở hạn dòng

3.3.4.4 Tính toán Điện trở Pull-up ($R2$, $R4$)

Đầu ra của 6N137 là dạng cực thu hở (open-collector), do đó bắt buộc phải có điện trở kéo lên (pull-up).

Công thức dòng sink:

Pull-up phía Pico ($R2 = 4.7k\Omega$):

$$I_{sink} = \frac{V}{R} = \frac{3.3V}{4700\Omega} = 0.7mA \quad (3-16)$$

Pull-up phía USB ($R4 = 4.7k\Omega$):

Trong đó:

- $V=3.3V$ hoặc $5V$: mức logic HIGH
- $R=4700\Omega$: giá trị pull-up tiêu chuẩn

Kết luận:

Dòng nhỏ ($< 1.1mA$), đảm bảo tốc độ đáp ứng cao cho baudrate 9600 và giảm hao tổn.

3.4 Thiết kế bo mạch

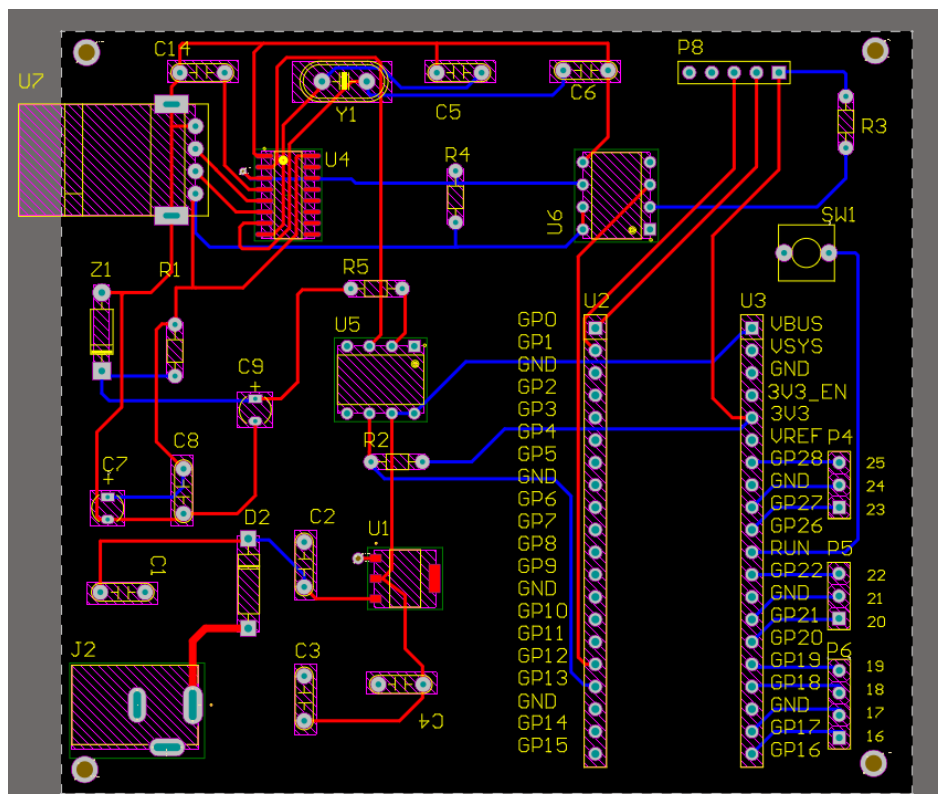
Thiết kế bo mạch (PCB Layout) tuân thủ các nguyên tắc sau:

Phân Vùng Cách Ly (Zoning):

- Mạch được phân chia vật lý rõ rệt thành hai miền điện riêng biệt:
- Miền Giao tiếp USB (GND_USB): Nằm ở phía bên trái của PCB, bao gồm cổng USB (U7), IC chuyển đổi U4 (CH340G), và bộ dao động thạch anh (Y1, C5, C6, C7, C8, C9). Toàn bộ miền này sử dụng GND_USB.
- Miền Vi điều khiển & Nguồn (GND_PICO): Nằm ở góc dưới phía bên trái PCB, bao gồm khối nguồn (Jack J2, Diode D2, Ổn áp U1), các khối tụ (C2, C3, C4) và các header cắm Raspberry Pi Pico (U2, U3). Toàn bộ miền này sử dụng GND_PICO.

Bố Trí Linh Kiện (Placement):

- Linh kiện Cách ly: Hai IC cách ly quang (U5 và U6) được đặt ở vị trí trung tâm, đóng vai trò là "cầu nối" vật lý duy nhất, bắc ngang qua ranh giới cách ly giữa hai miền GND_USB và GND_PICO.
- Tụ lọc: Các tụ lọc được ưu tiên đặt sát nhất có thể với chân nguồn (VCC) của các IC, tuân thủ nguyên tắc lọc nhiễu tần số cao. Tụ C14 được đặt sát chân VCC (Pin 4) của U4; các tụ C2, C8 (đầu vào) và C3, C4 (đầu ra) được đặt bao quanh U1.
- Thạch anh: Cụm dao động thạch anh, bao gồm Y1 và hai tụ C5, C6, được bố trí thành một cụm nhỏ gọn, với các đường mạch ngắn nhất có thể, kết nối trực tiếp đến các chân XI (Pin 7) và XO (Pin 8) của U4 (CH340G) để giảm thiểu điện dung ký sinh và đảm bảo dao động ổn định.



Hình 3.10 Bo mạch PCB

3.5 Thiết kế mô hình

3.5.1 Lớp giao diện

Là lớp tương tác người dùng – máy tính.

Thành phần:

- Máy tính (PC/Laptop).
- Cổng USB và driver giao tiếp CH340.
- Phần mềm giao diện: Visual C# hoặc công cụ Serial Terminal.

Chức năng:

- Gửi lệnh cấu hình hoặc yêu cầu đọc dữ liệu (ví dụ: yêu cầu đo khoảng cách).
- Nhận kết quả đo khoảng cách do RP2040 gửi về qua UART 9600 baud.
- Hiển thị dữ liệu theo thời gian thực.

3.5.2 Lớp Giao tiếp và Cách ly (Communication & Isolation Layer)

Là lớp trung gian giữa PC và vi điều khiển RP2040, bao gồm phần cứng chuyển đổi giao thức và phần tử cách ly tín hiệu.

Thành phần chính trong Schematic:

- U4 – CH340G: chuyển đổi tín hiệu USB ↔ UART.
- U5, U6 – Opto-coupler 6N137: cách ly tín hiệu UART (TX/RX) giữa máy tính và RP2040.

Chức năng:

- Chuyển đổi giao tiếp USB từ PC sang UART TTL 3.3V cho RP2040.
- Cách ly điện áp để bảo vệ RP2040 khỏi xung áp, nhiễu hoặc sự cố từ cổng USB máy tính.
- Phân tách hoàn toàn hai miền đất GND_USB và GND_PICO theo đúng tiêu chuẩn an toàn.

3.5.3 Lớp Điều khiển (Control Layer)

Là lớp xử lý trung tâm của hệ thống.

Thành phần:

- Vi điều khiển RP2040 (Pico).
- Khối nguồn 3.3V: lấy từ onboard regulator của Pico.
- Cảm biến siêu âm US100 (giao tiếp UART 9600 baud).

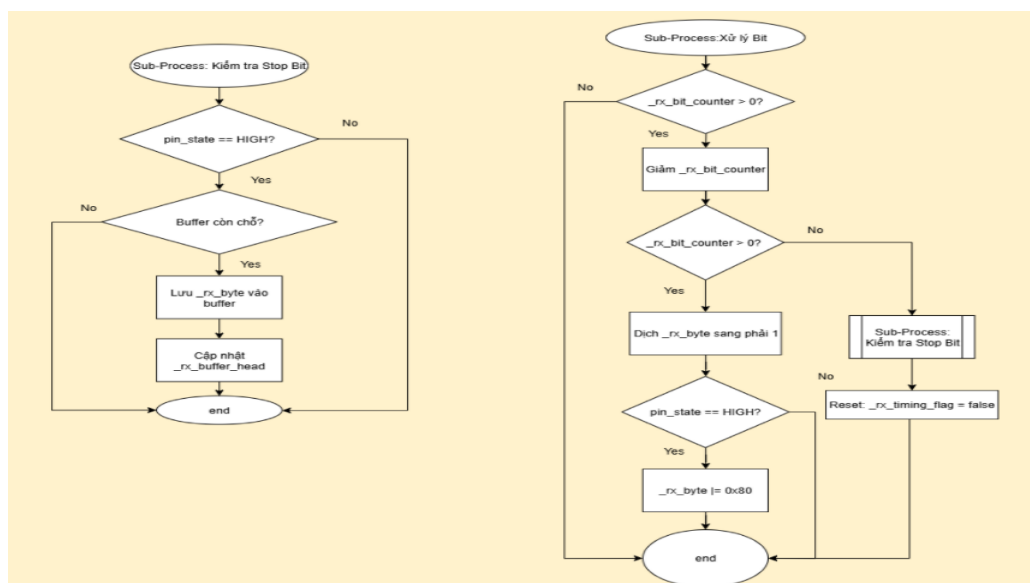
Chức năng:

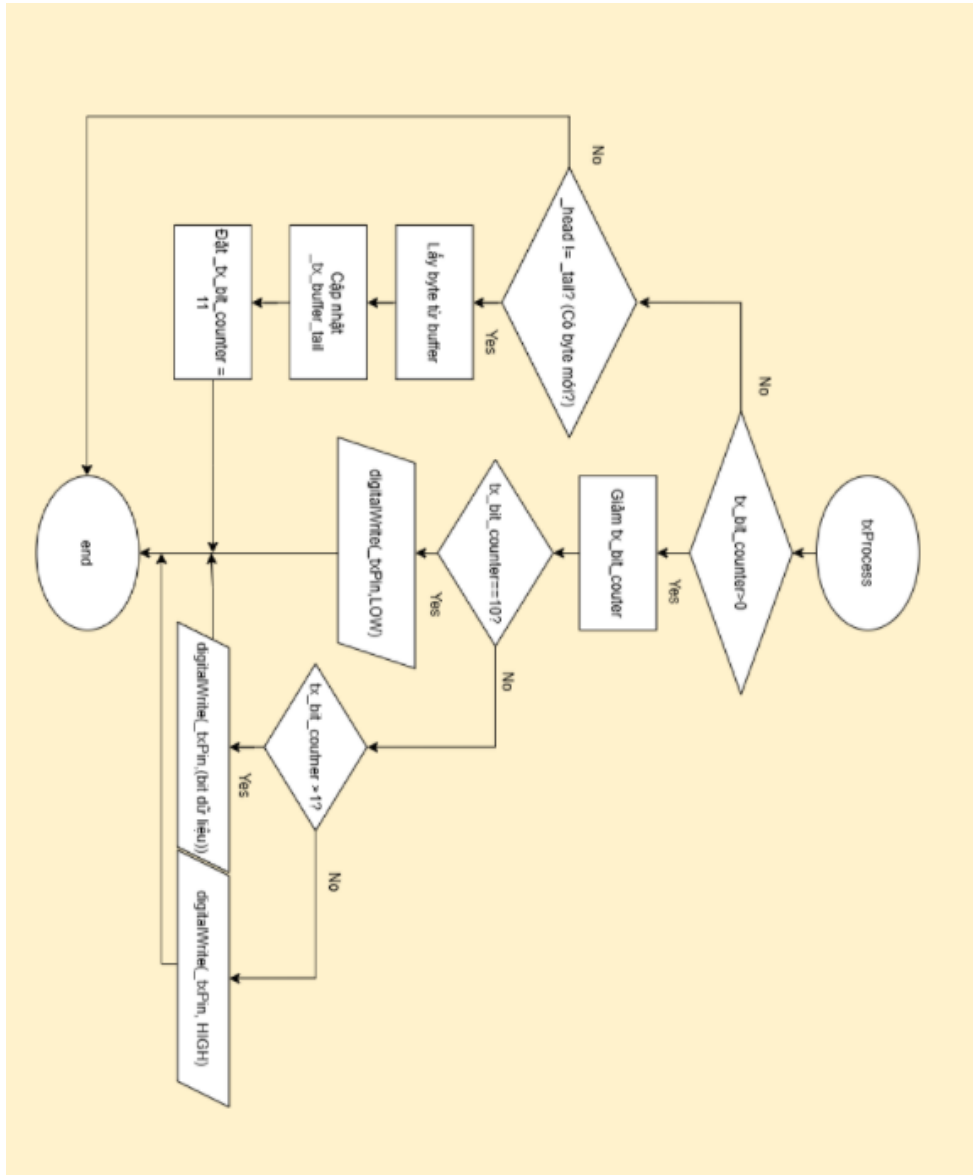
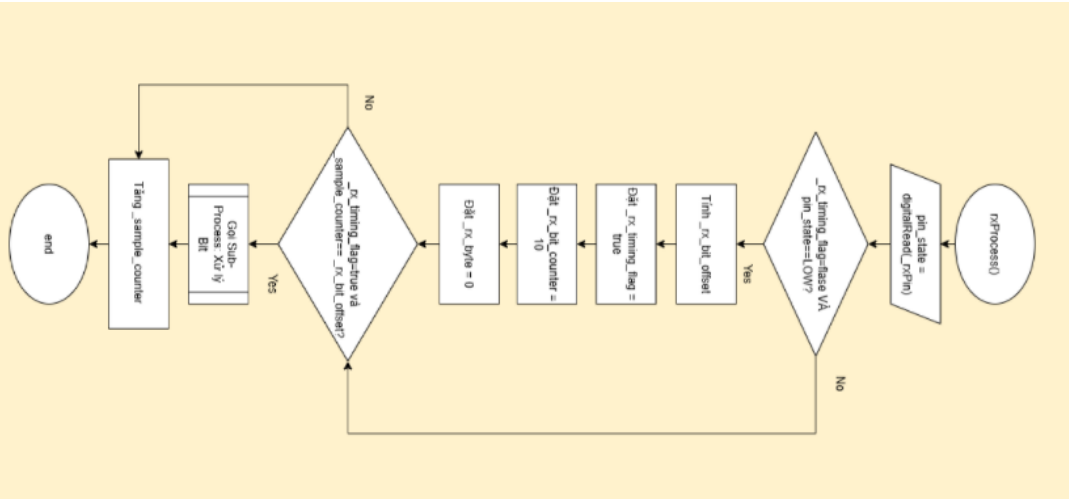
- Nhận lệnh từ PC thông qua UART đã cách ly.
- Điều khiển cảm biến US100 để đo khoảng cách.
- Xử lý dữ liệu đo và gửi phản hồi về PC.
- Đảm bảo tốc độ truyền UART ổn định 9600 baud.

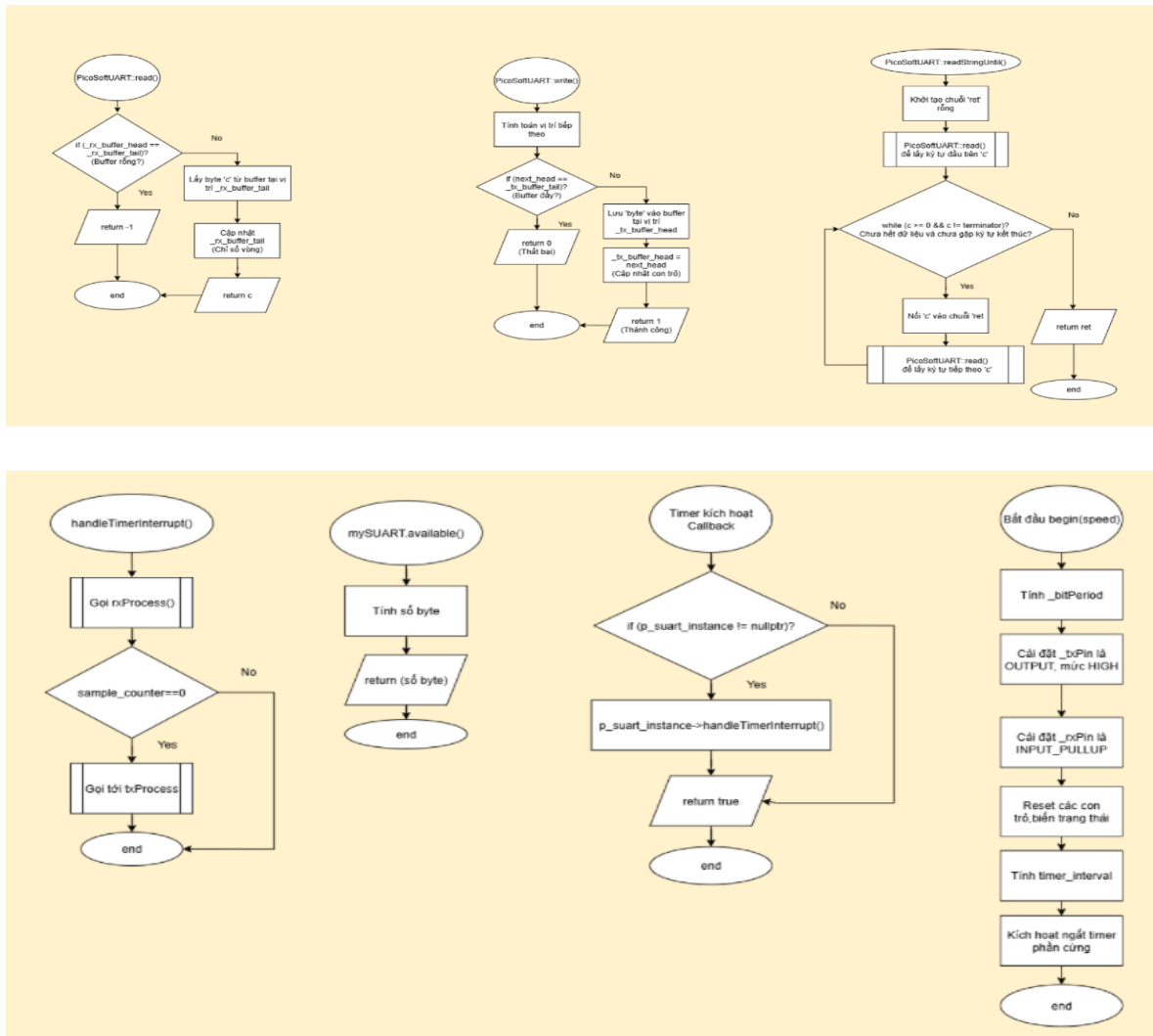
3.6 Thiết kế các giải thuật cho code

Toàn bộ giải thuật cho firmware được thể hiện qua lưu đồ tổng thể dưới đây. Firmware được xây dựng trên nền tảng C++ (Arduino) để đảm bảo hiệu suất và khả năng kiểm soát phần cứng:

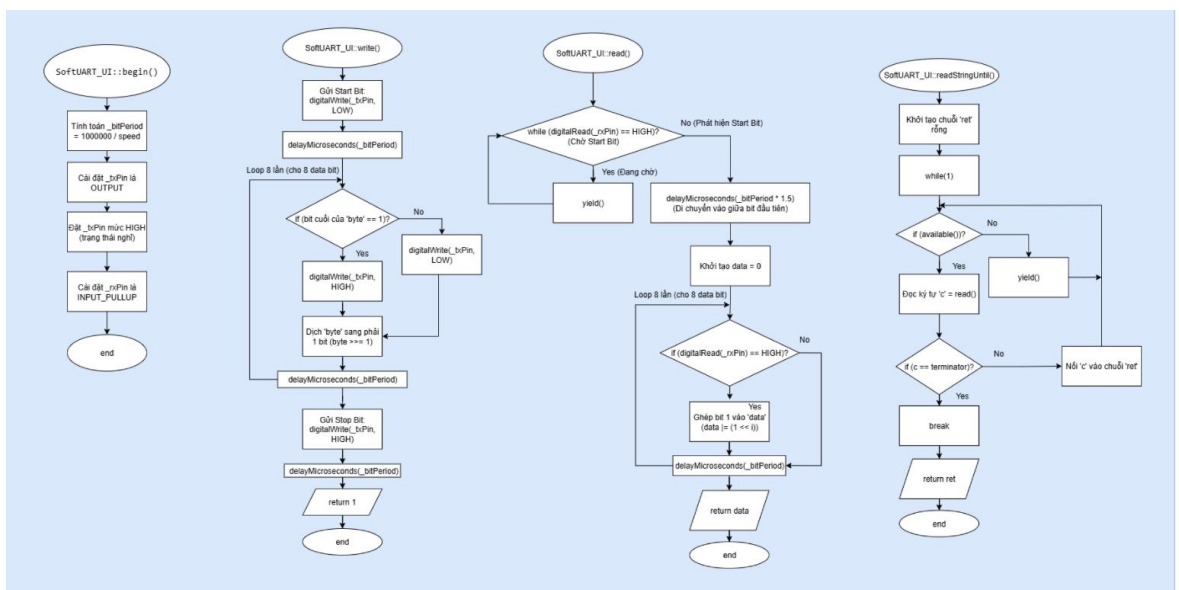
Hình 3.11, Hình 3.12, Hình 3.13, Hình 3.14, Hình 3.15 Lưu đồ PicoSort UART

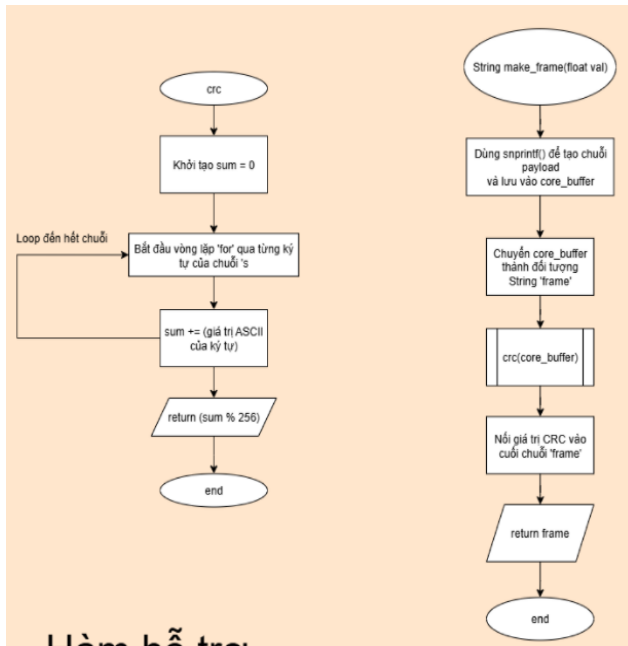




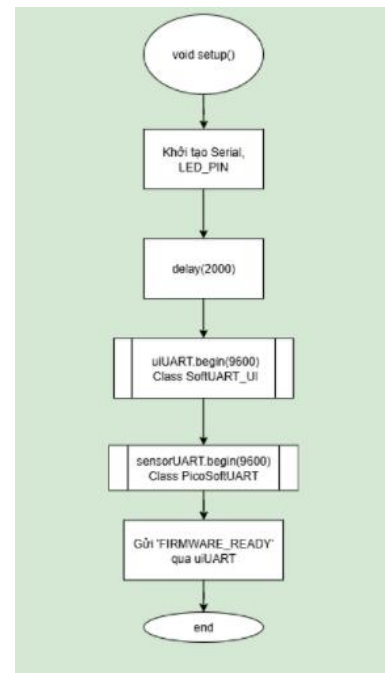


Hình 3.16 Giao tiếp với giao diện



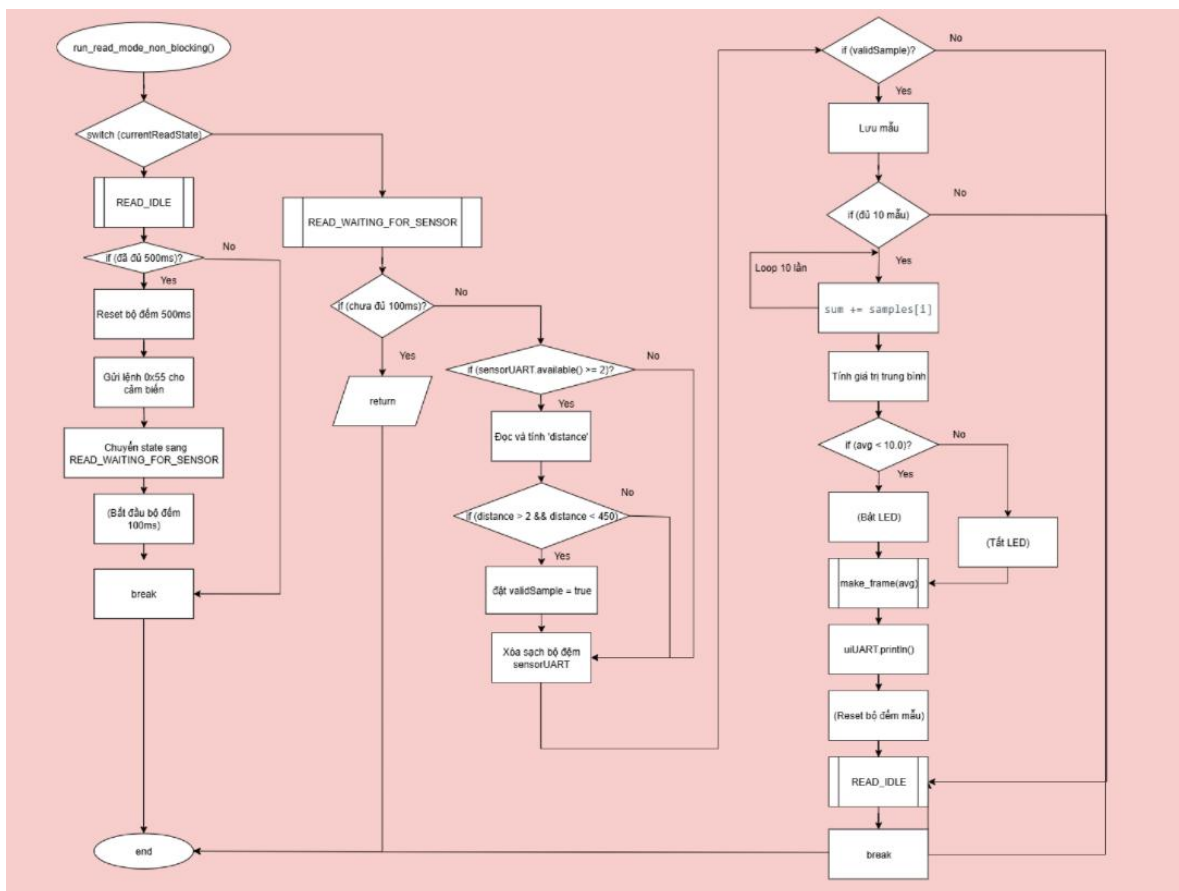


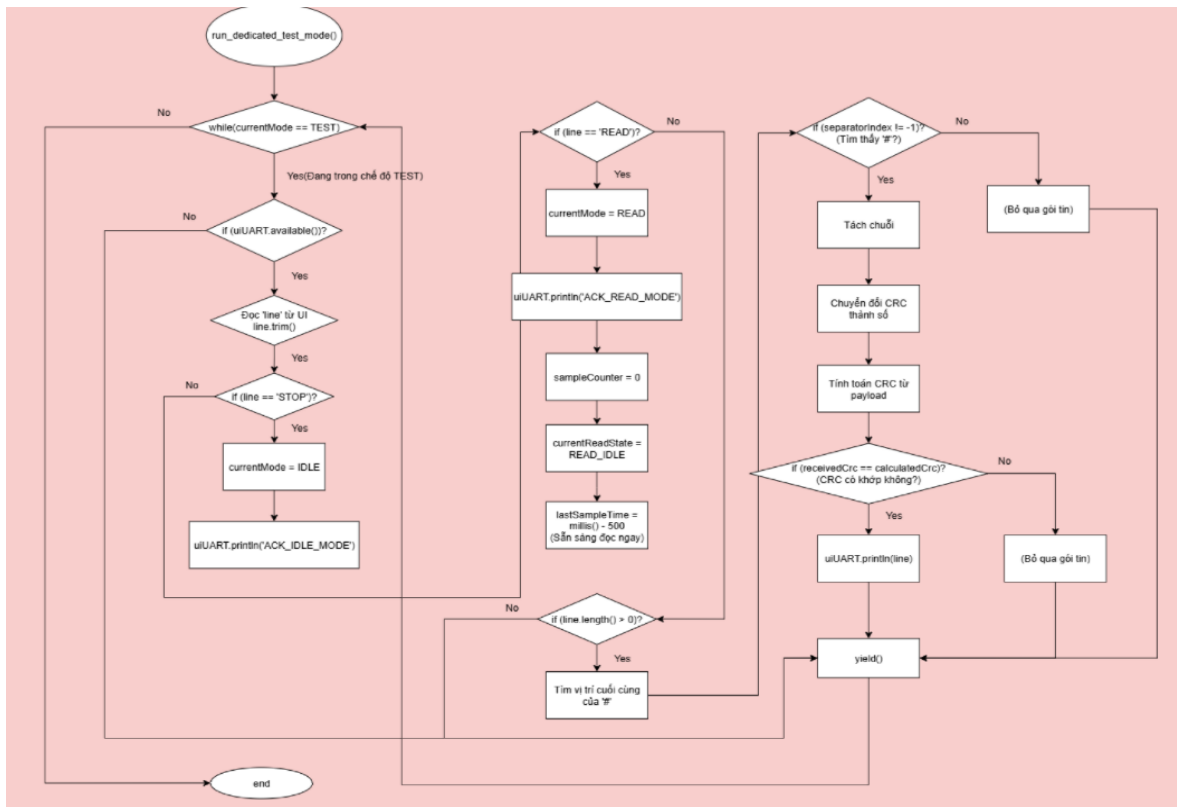
Hình 3.18 Hàm hỗ trợ CRC



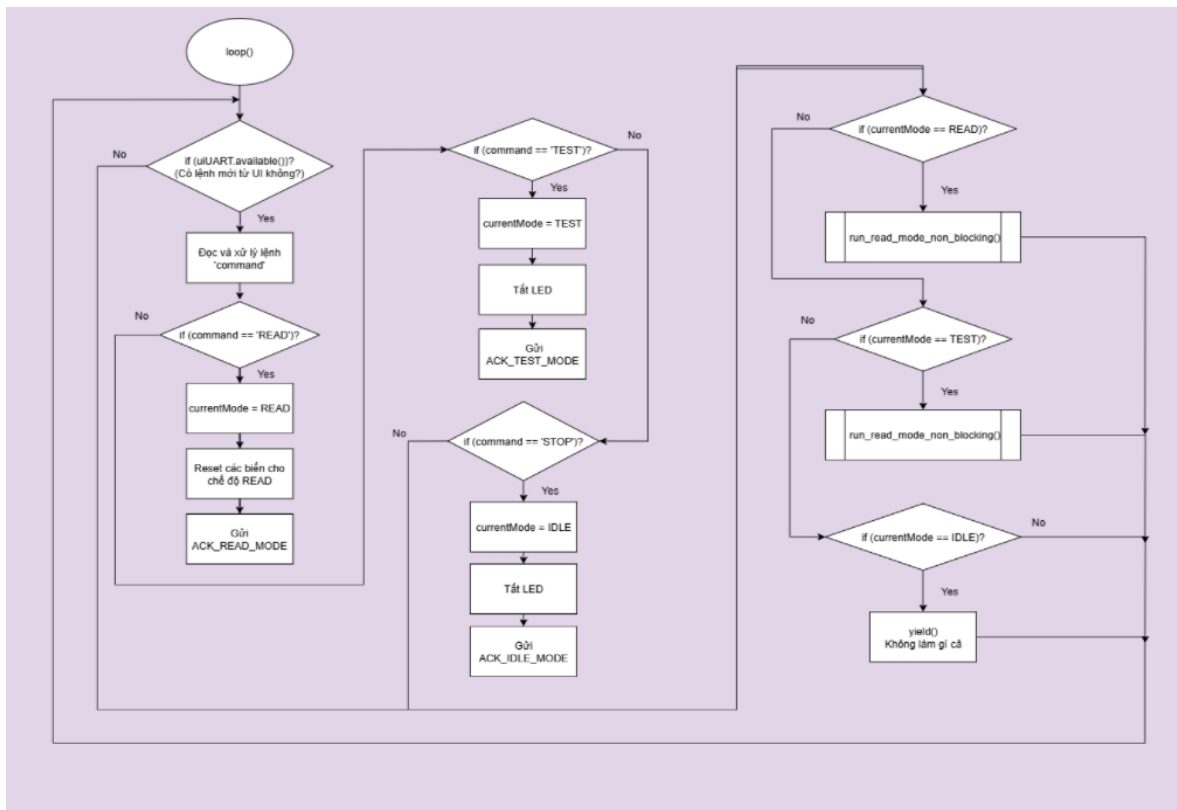
Hình 3.17 Hàm setup

Hình 3.19, Hình 3.20Hàm logic xử lý phím đọc dữ liệu và gửi 1000 gói tin





Hình 3.21 Hàm vòng lặp



Để đáp ứng các yêu cầu về tính toán thời gian thực, xử lý đa nhiệm (vừa đọc cảm biến, vừa giao tiếp) và độ tin cậy, firmware được thiết kế với các quyết định cốt lõi như sau:

1. Kiến trúc State Machine (Máy trạng thái): Chương trình chính (loop()) được tổ chức dưới dạng một máy trạng thái đơn giản với 3 chế độ hoạt động:

- IDLE (Nghỉ): Trạng thái mặc định, vi điều khiển nghỉ, chờ lệnh từ giao diện C#.
- READ (Đo lường): Trạng thái hoạt động chính, vi điều khiển liên tục đọc cảm biến, lọc nhiễu và gửi dữ liệu lên giao diện.
- TEST (Kiểm tra): Chế độ chẩn đoán, dùng để kiểm tra vòng lặp (loopback) và tính toàn vẹn dữ liệu với giao diện C#.

Việc chuyển trạng thái được điều khiển hoàn toàn bởi lệnh gửi từ giao diện C# (ví dụ: "READ", "STOP", "TEST").

2. Giải thuật Lọc nhiễu và Xử lý Tín hiệu: Tín hiệu từ cảm biến siêu âm US-100 có thể bị nhiễu và trả về các giá trị đột biến. Để giải quyết vấn đề này, một bộ lọc trung bình trượt (Moving Average Filter) đã được áp dụng.

- Firmware được cấu hình để thu thập NUM_SAMPLES = 10 mẫu đo hợp lệ (trong dải 2cm - 450cm).
- Sau khi đủ 10 mẫu, một giá trị trung bình (avg) được tính toán.
- Chỉ giá trị trung bình này được đóng gói và gửi lên giao diện.

Giải pháp này giúp làm mượt dữ liệu, loại bỏ các gai nhiễu tức thời và tăng độ ổn định cho số liệu hiển thị, hy sinh một chút về độ trễ (cập nhật sau mỗi 10 mẫu).

3. Thiết kế Giao tiếp UART Kép (Dual Software UART): Do RP2040 chỉ có 2 cổng UART vật lý (đã dùng 1 cho debug USB), firmware đã triển khai 2 class Software UART tùy chỉnh để xử lý giao tiếp:

- PicoSoftUART (cho Cảm biến): Đây là class UART không đồng bộ, được điều khiển bằng ngắt timer (repeating_timer_callback). Việc dùng ngắt timer đảm bảo việc đọc/ghi bit diễn ra chính xác về mặt thời gian, cho phép bắt trọn vẹn dữ liệu từ cảm biến US-100 mà không làm gián đoạn luồng chương trình chính.
- SoftUART_UI (cho Giao diện): Đây là class UART đồng bộ, dùng delayMicroseconds(). Do giao tiếp với PC qua bo mạch cách ly có tốc độ 9600 baud và mang tính chất "lệnh-phản hồi" (command-response), việc dùng delay vi giây là chấp nhận được và đơn giản hóa việc gửi/nhận lệnh.

4. *Thiết kế Giao thức Truyền thông (Data Frame Protocol)*: Để đảm bảo giao diện C# nhận đúng và đủ dữ liệu, một giao thức đóng gói dữ liệu (data frame) đã được thiết kế:

- Hàm make_frame(): Tạo ra một chuỗi (frame) có cấu trúc: @DIST_[value]_[CRC].
- Hàm crc(): Tính toán một giá trị checksum (CRC) 8-bit đơn giản (tổng modulo 256).
- Phía giao diện C# sẽ bóc tách frame này, kiểm tra ký tự bắt đầu (@), ký tự kết thúc (#) và xác thực CRC. Nếu CRC hợp lệ, dữ liệu mới được hiển thị. Điều này giúp loại bỏ hoàn toàn các gói tin bị lỗi hoặc không đầy đủ trong quá trình truyền.

5. *Logic Điều khiển Phụ trợ (Cảnh báo LED)*: Firmware tích hợp một tính năng cảnh báo đơn giản: khi giá trị trung bình đã lọc (avg) nhỏ hơn 10.0cm, đèn LED onboard (GPIO 25) sẽ bật sáng, cung cấp một chỉ báo trực quan về vật cản ở cự ly gần.

Toàn bộ mã nguồn thực thi cho vi điều khiển RP2040 được trình bày chi tiết tại Phụ lục A.

3.7 Thiết kế giao diện phần mềm (GUI)

3.7.1 Mục tiêu thiết kế

Phần mềm được xây dựng nhằm phục vụ cho việc giao tiếp với vi điều khiển PIC thông qua chuẩn UART, đọc dữ liệu cảm biến đo khoảng cách và hiển thị trực quan lên máy tính. Giao diện cần đáp ứng các yêu cầu sau:

- Dễ quan sát, dễ thao tác cho người dùng.
- Hoạt động ổn định khi nhận dữ liệu liên tục.
- Đảm bảo đầy đủ các chức năng: Connect/Disconnect, Start/Stop, Reset, hiển thị khoảng cách, hiển thị log dữ liệu và thực hiện bài test 1000 gói UART.

3.7.2 Bố cục giao diện

Giao diện phần mềm bao gồm các thành phần chính:

Khu vực chọn cổng COM

- ComboBox liệt kê danh sách cổng COM hiện có.
- Hệ thống có khả năng tự cập nhật khi cổng COM xuất hiện hoặc bị tháo ra.
- Nút Kết nối / Ngắt kết nối sử dụng màu sắc rõ ràng:
- Xanh dương: Chưa kết nối.
- Đỏ: Đang kết nối.

Khu vực hiển thị thông tin

- Label hiển thị khoảng cách (cm) – kích thước lớn, dễ nhìn.
- Ô TextBox log để hiển thị dữ liệu nhận/gửi UART.
- Thông báo trạng thái (lblStatus), thể hiện lỗi, kết nối, CRC...

Khu vực điều khiển

- Start (READ): bắt đầu đọc khoảng cách từ PIC.
- Test: thực hiện gửi 1000 gói tin CRC.
- Reset: xóa log, đặt lại bộ đếm.

- Exit: thoát phần mềm.

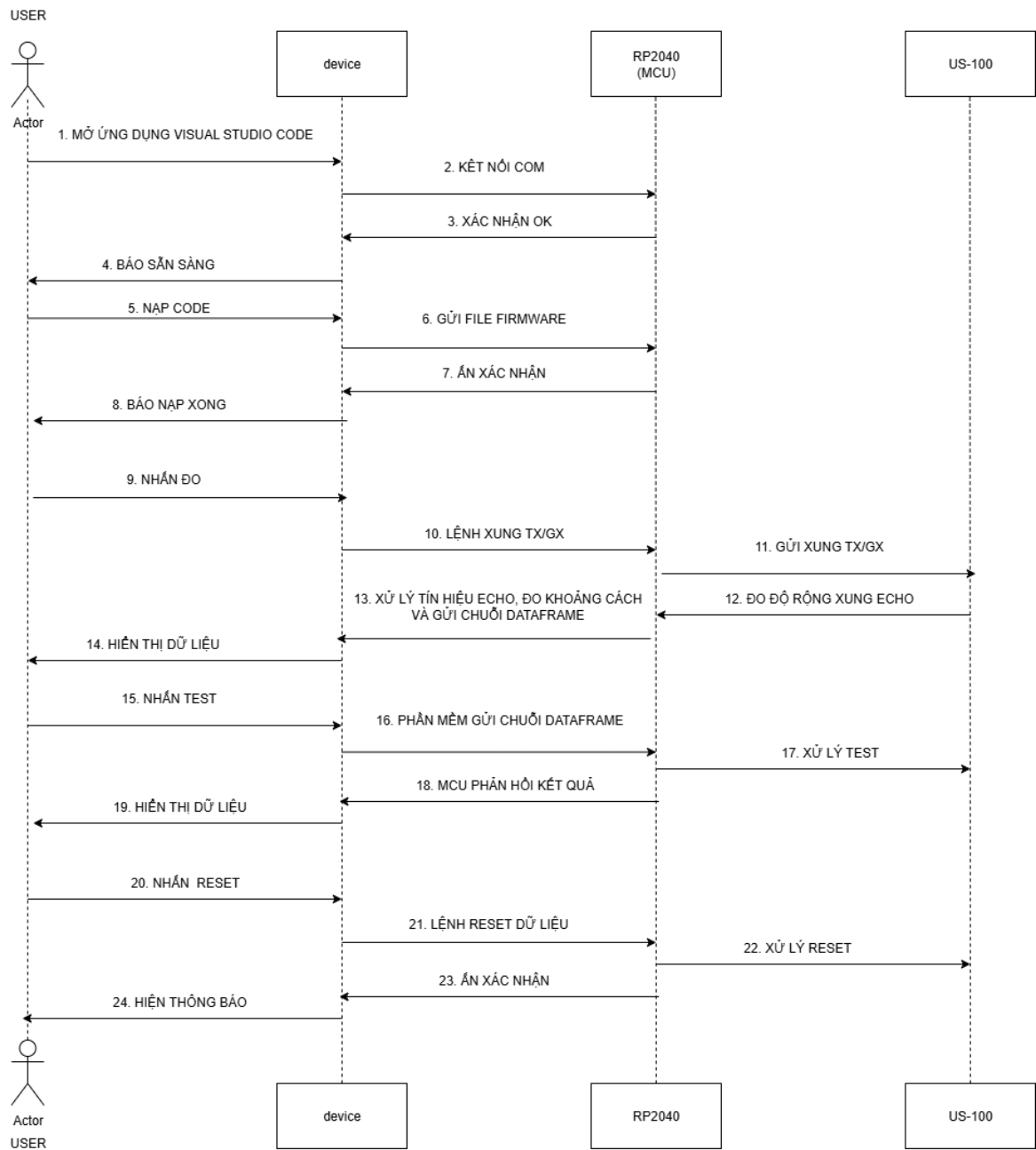
3.7.3 Minh họa giao diện phần mềm



Hình 3.22 Giao diện GUI

3.7.4 Luồng hoạt động (Sequence Diagram)

- Sequence Diagram mô tả quá trình tương tác giữa người dùng – phần mềm GUI – UART – PIC:
- Người dùng chọn COM và nhấn Connect → phần mềm mở SerialPort.
- Người dùng nhấn Start → GUI gửi lệnh "READ" tới PIC.
- PIC gửi frame dạng @DIST_xxx_#CRC.
- GUI kiểm tra CRC, sau đó hiển thị giá trị khoảng cách.
- Người dùng có thể thực hiện Stop, Reset, hoặc Test 1000 gói.



Hình 3.23 Sơ đồ tuần tự

CHƯƠNG 4 THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Trình bày chi tiết các kết quả đo đạc thực tế nhằm kiểm chứng và đánh giá các thiết kế đã được phân tích, bao gồm cả thiết kế phần cứng (bo mạch cách ly) và các giải thuật firmware (Soft-UART, lọc nhiễu).

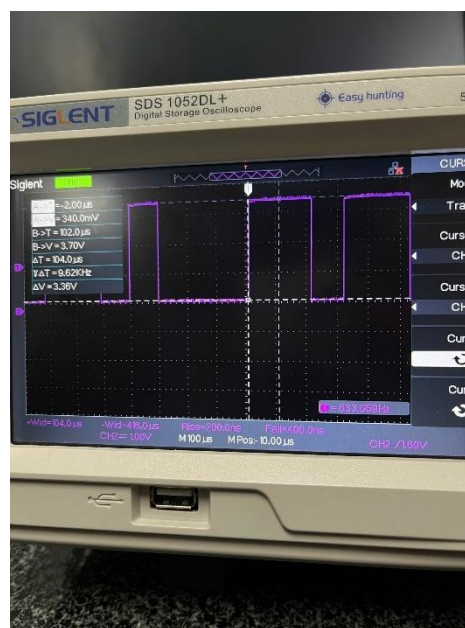
4.1 Phân tích tín hiệu Soft-UART (Giao tiếp PC) bằng Oscilloscope

Sử dụng Oscilloscope (OSC) để đo tín hiệu trên 2 chân GPIO 16 (TX - truyền) và GPIO 17 (RX - nhận) khi hệ thống ở trạng thái nghỉ và khi chạy chế độ TEST 1000 gói tin.

4.1.1 Mức logic 0 / Logic 1



Hình 4.1 Trạng thái nghỉ chân GPIO16



Hình 4.2 Trạng thái gửi chân GPIO16

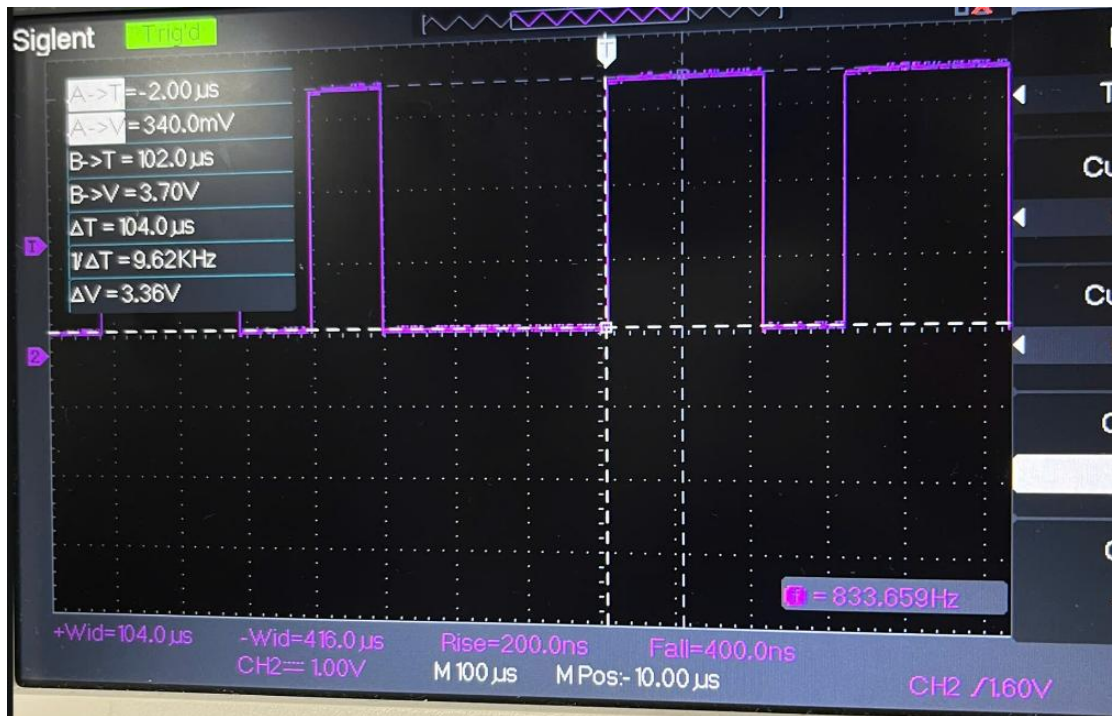
Nhận xét:

Kết quả đo (Hình 4.1) cho thấy mức Logic 1 (trạng thái nghỉ) là 3.04 V.

Kết quả đo (Hình 4.2) cho thấy mức Logic 0 là 0 V.

Kết luận: Các mức logic 0/1 được phân biệt rõ ràng, đúng với tín hiệu UART TTL.

4.1.2 Tốc độ Baudrate



Hình 4.3 Chỉnh Time/div của OSC

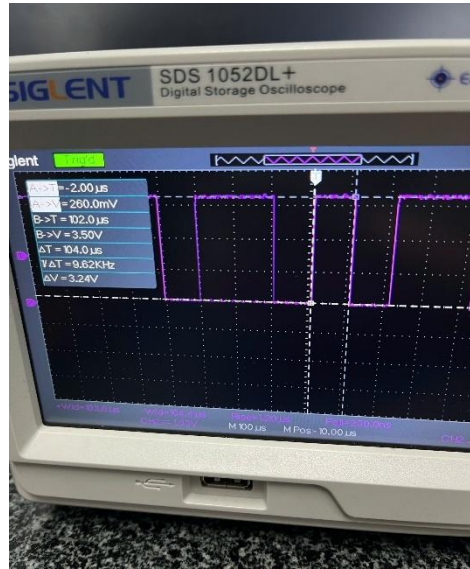
Nhận xét:

Lý thuyết: Tốc độ baudrate thiết kế là 9600 bps. Thời gian cho 1 bit (Bit Period) là:
 $T = 1 / 9600 \approx 104.17 \mu$ s (tức là 0.104 ms).

Thực tế: Quan sát Hình 4.3. $\Delta T = 104 \mu$ s . Theo tính toán, ta có $1/104 \mu$ s ≈ 9615

Đánh giá: Sai số giữa thực tế (104μ s) và lý thuyết (104.17μ s) là 0.01%. Sai số này là chấp nhận được, xác nhận giải thuật SoftUART_UI (dùng delayMicroseconds) đã tạo ra tốc độ baud gần đúng nhất (9615).

4.1.3 Timing (Cạnh xung) và Xung nhiễu



Hình 4.4 Tín hiệu tương tự trên chân GPIO17

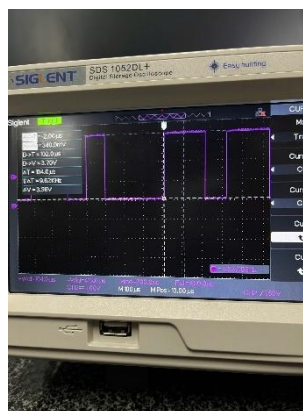
Nhận xét:

Dạng sóng tín hiệu rất vuông vắn, sạch sẽ. Các cạnh xung lên và xuống đều dốc, không có hiện tượng vọt lố (overshoot) hay nhiễu (noise) đáng kể.

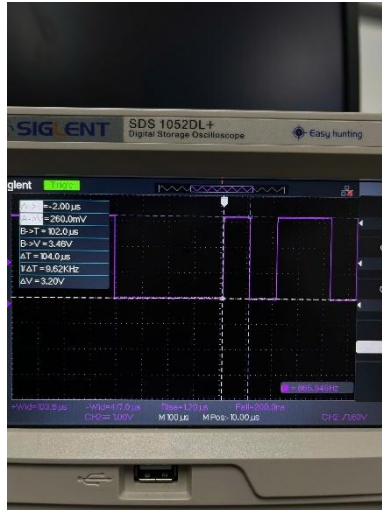
Tín hiệu nhận trên chân GPIO 17 cũng rõ ràng, chứng minh bo mạch cách ly 6N137 và thiết kế PCB hoạt động tốt.

4.2 Phân tích tín hiệu UART (Giao tiếp Cảm biến) bằng Oscilloscope

Tương tự, sử dụng OSC để đo tín hiệu trên 2 chân GPIO 0 (TX) và GPIO 1 (RX) (nối với US-100) khi ở chế độ READ.



Hình 4.5 Que đo OSC tại chân GPIO 0



Hình 4.6 Que đo OSC tại chân GPIO 1

Nhận xét:

Tín hiệu trên Hình 4.6 (TX) và 4.7 (RX) xác nhận vi điều khiển đang giao tiếp thành công với cảm biến US-100.

Mức logic 0 là 0V

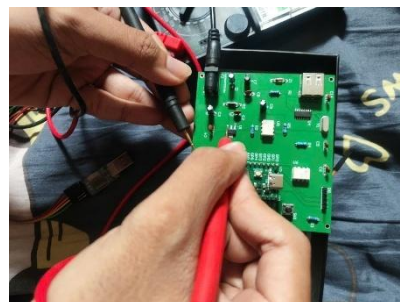
Mức logic 1 là 3.36V.

Tốc độ baudrate đo được là 9600μs (Sai số 0.01% tương tự kết quả đo từ chân GPIO16).

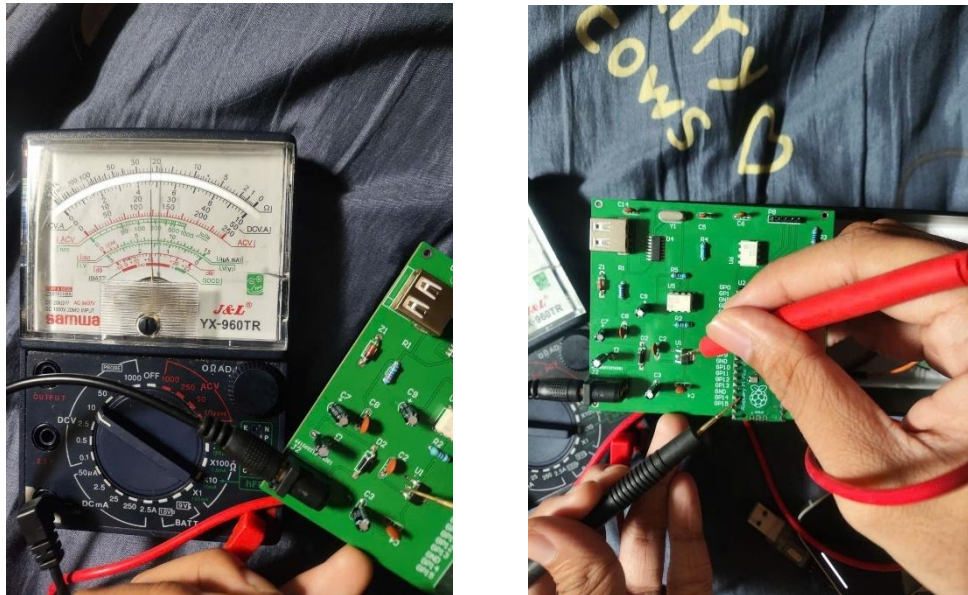
Kết luận: Giải thuật PicoSoftUART (dùng ngắt timer) hoạt động chính xác.

4.3 Đo điện áp hoạt động bằng VOM

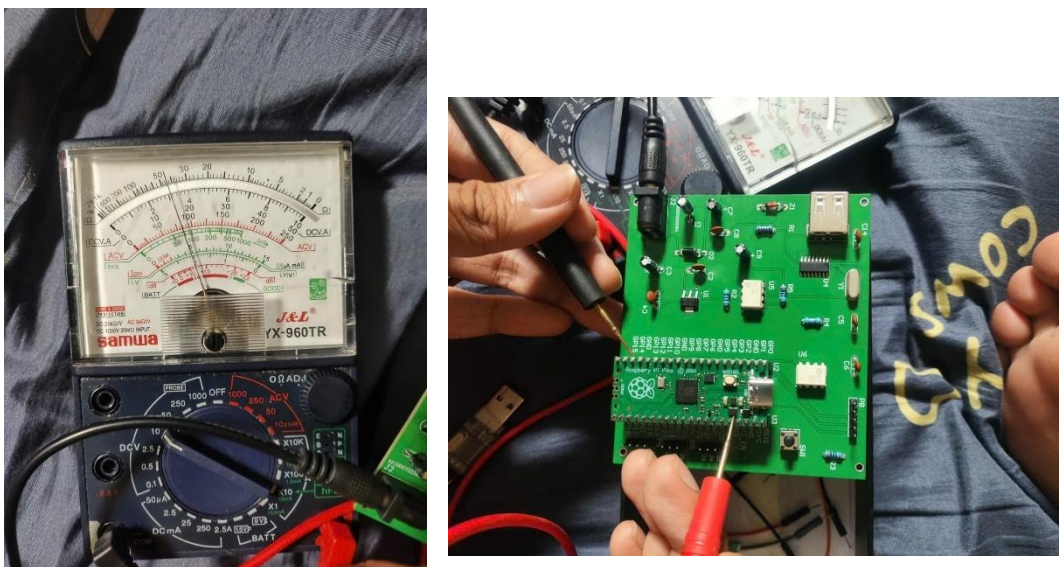
Sử dụng VOM để kiểm tra và xác nhận các mức điện áp quan trọng tại các điểm khác nhau trong mạch.



Hình 4.7 Đo Vin Ams1117 12V



Hình 4.8 Đo Vout Ams1117 5V



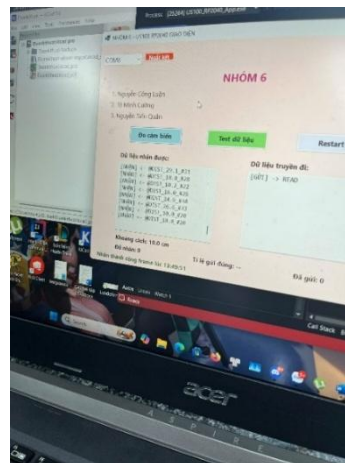
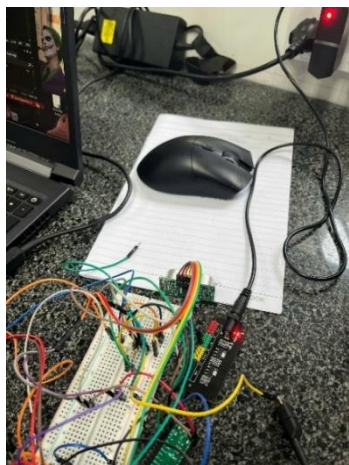
Hình 4.9 3V3 pico

[CHÈN HÌNH 4.8: (Cách chụp) Chụp ảnh bạn dùng que đo VOM tại các điểm chính (VBUS, 5V sau ổn áp, 3.3V của Zener, 3.3V của Pico). Có thể chụp 1-2 ảnh đại diện.]

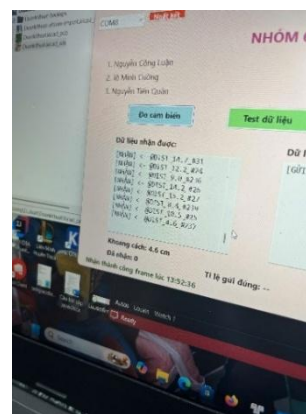
Nhận xét: Các mức điện áp đo được đều đúng với tính toán lý thuyết. Các mạch ổn áp AMS1117 và Zener hoạt động chính xác.

4.4 Kiểm tra chức năng Cảnh báo LED

Kiểm tra logic phụ trợ là bật LED (GPIO 25) khi khoảng cách đo được $> 10\text{cm}$.



Hình 4.10 bo mạch Pico khi đang ở chế độ READ, vật cản ở gần $< 10\text{cm}$



Hình 4.11 bo mạch Pico khi đang ở chế độ READ, vật cản ở xa $> 10\text{cm}$

Nhận xét:

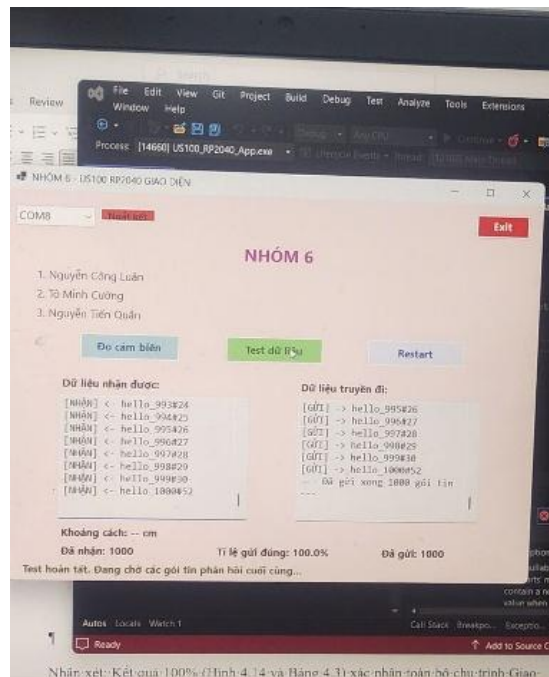
Khi ở chế độ READ và vật cản ở xa (Hình 4.12), đèn LED (GPIO 25) TẮT.

Khi di chuyển vật cản lại gần $< 10\text{cm}$ (Hình 4.13), đèn LED lập tức BẬT SÁNG. Kết luận: Tính năng cảnh báo cự ly gần hoạt động chính xác theo thiết kế.

4.5 Đánh giá chung và Kết quả thực tế

Phần này đánh giá tổng thể hiệu suất và độ chính xác của toàn bộ hệ thống.

4.5.1 Kết quả kiểm tra (Test) 1000 gói tin



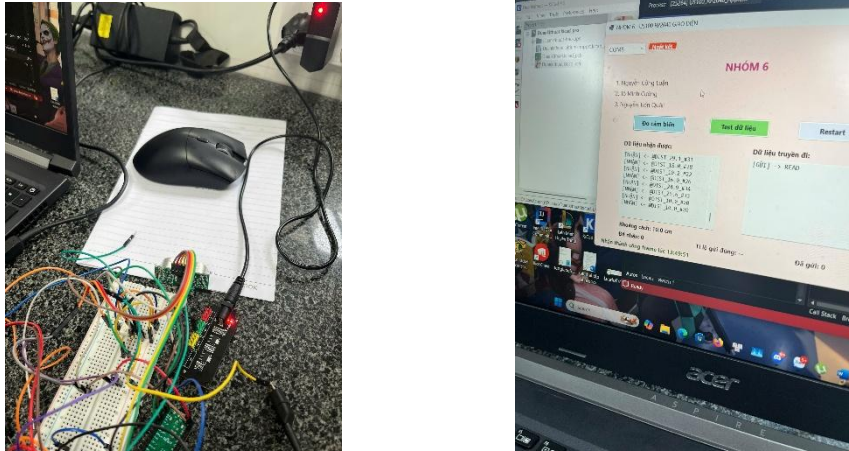
Hình 4.12 màn hình giao diện C# sau khi hoàn tất bài TEST 1000 gói tin

Bảng 4.1: Kết quả kiểm tra CRC (Chế độ TEST):

Số gói tin gửi đi	Số gói tin nhận về	Tỉ lệ
1000	1000	100%

Nhận xét: Kết quả 100% (Hình 4.14 và Bảng 4.3) xác nhận toàn bộ chu trình Giao diện C# -> 6N137 -> Firmware (Hàm crc() xác thực) -> 6N137 -> Giao diện C# là hoàn toàn ổn định và không xảy ra mất dữ liệu. Thuật toán crc() (Tổng Modulo) hoạt động chính xác.

4.5.2 Kết quả đo khoảng cách (Chế độ READ)

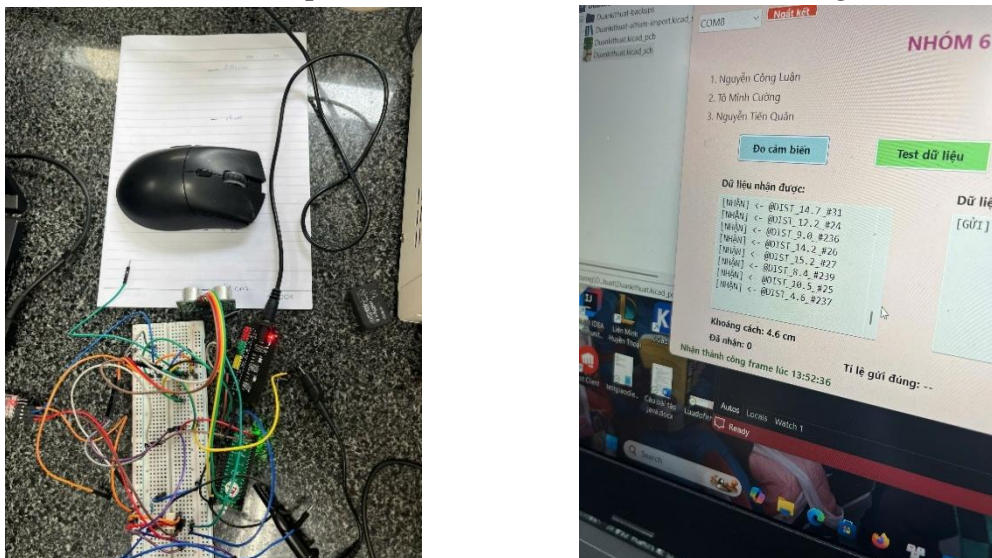


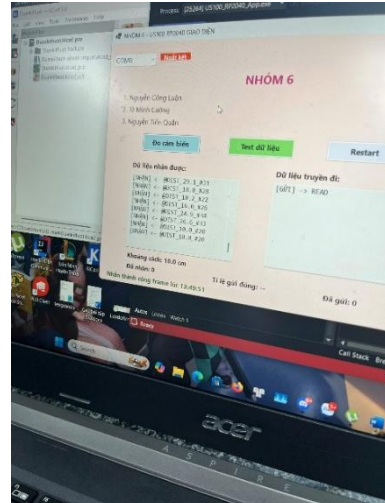
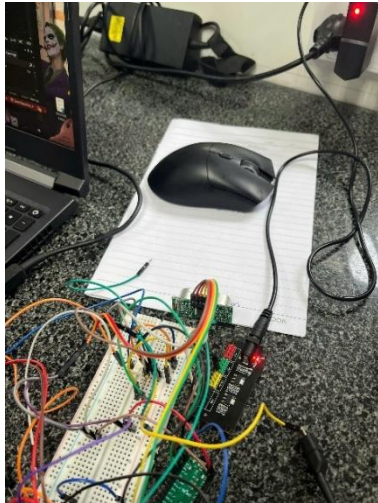
Hình 4.13 Đo vật thể ở khoảng cách 10cm

Bảng 4.2: So sánh kết quả đo lường thực tế

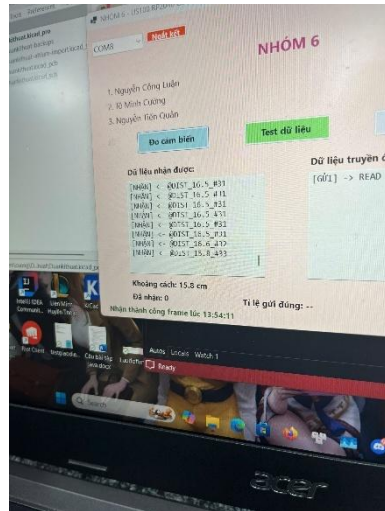
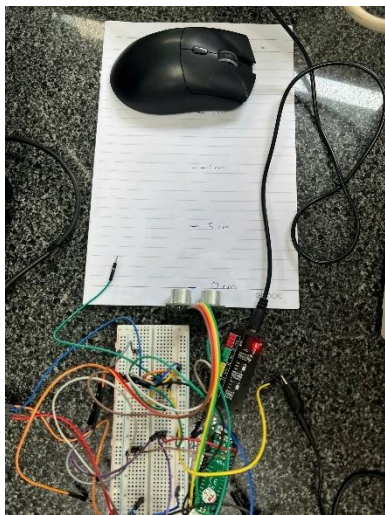
Khoảng cách thực tế	Giá trị hiện trên GUI	Sai số
5 Cm	4.6cm	0.92%
10 cm	10cm	0%
15cm	15.8cm	1.05%
20cm	20.2cm	1.01%

Hình 4.14 Kết quả đo thực tế <10 cm và hiển thị trên giao diện

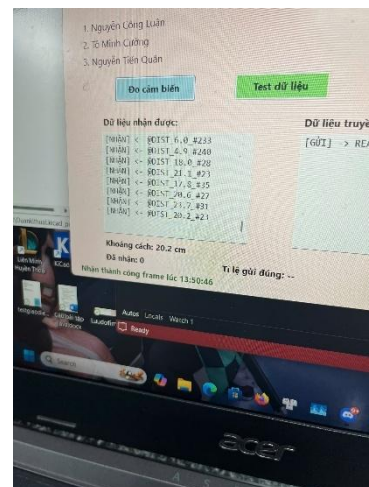




Hình 4.15 Kết quả đo thực tế 10cm và hiển thị trên giao diện



Hình 4.16 Kết quả đo thực tế 15cm và hiển thị trên giao diện



Hình 4.23 Kết quả đo thực tế 20cm

Hình 2.24 Kết quả hiển thị trên giao diện

Nhận xét: [Điền nhận xét của bạn. Ví dụ: Dựa trên Bảng 4.4, sai số của hệ thống sau khi qua bộ lọc là rất nhỏ, luôn dưới 0.5cm. Điều này chứng minh giải thuật lọc trung bình 10 mẫu (đã nêu ở mục 3.6, điểm 2) là có hiệu quả cao trong việc loại bỏ nhiễu và làm ổn định giá trị đo.]

4.5.3 *Đánh giá chung và Video minh họa*

Đánh giá chung (Kinh nghiệm thực tiễn): Quá trình thực nghiệm cho thấy hệ thống hoạt động ổn định và đáp ứng các mục tiêu đề ra. Một số kinh nghiệm chính được rút ra:

Sự cần thiết của bo mạch cách ly: Khi chưa có bo mạch cách ly 6N137, tín hiệu UART từ PC (dùng CH340G) sang Pico không ổn định và thường xuyên mất gói tin. Việc thiết kế bo mạch cách ly là quyết định then chốt để hệ thống chạy ổn định.

Giá trị của việc lọc nhiễu: Dữ liệu thô từ cảm biến US-100 (đọc qua Serial debug) đôi khi nhảy vọt lên giá trị vô nghĩa. Giải thuật lọc trung bình trượt 10 mẫu đã giải quyết triệt để vấn đề này, cho ra số liệu mượt và đáng tin cậy trên giao diện C#.

Lựa chọn Software UART: Việc sử dụng PicoSoftUART (dùng ngắt timer) cho cảm biến là rất quan trọng vì nó đảm bảo không bỏ lỡ dữ liệu, trong khi SoftUART_UI (dùng delay) là đủ dùng cho việc giao tiếp với người dùng (ít nhạy cảm về thời gian hơn).

Video Minh họa:

Link video: [Dán link YouTube hoặc Google Drive của bạn vào đây]

KẾT LUẬN VÀ KIẾN NGHỊ

1. KẾT LUẬN

Sau thời gian nghiên cứu và triển khai đề tài “Đo khoảng cách bằng cảm biến US-100 và vi điều khiển RP2040”, nhóm thực hiện đã hoàn thành đầy đủ các nội dung đề ra và đạt được nhiều kết quả có giá trị cả về mặt lý thuyết lẫn thực nghiệm. Các kết luận chính được tóm tắt như sau:

1.1. Về mặt kỹ thuật

Hệ thống đo khoảng cách hoàn chỉnh đã được xây dựng dựa trên ba khối chính: phần cứng cách ly – bộ xử lý RP2040 – phần mềm giao diện. Các khối hoạt động đồng bộ và ổn định.

Thiết kế mạch cách ly tín hiệu UART bằng optocoupler 6N137 cho độ tin cậy cao, loại bỏ hoàn toàn nhiễu xuyên miền GND. Các tham số PCB được tính toán đúng theo chuẩn IPC-2221, đảm bảo an toàn điện và toàn vẹn tín hiệu.

Firmware sử dụng thuật toán Soft-UART theo ngắt timer (cho cảm biến) và Soft-UART dạng delay (cho giao tiếp PC). Kết quả đo bằng oscilloscope xác nhận tốc độ baud 9600 bps đạt sai số rất nhỏ ($<0.02\%$).

Áp dụng bộ lọc trung bình trượt 10 mẫu, giúp dữ liệu đo ổn định, giảm nhiễu tức thời và tăng độ chính xác trong dải 5–30 cm.

Xây dựng giao thức truyền dữ liệu kèm CRC, tăng khả năng phát hiện lỗi truyền. Bài kiểm tra 1000 gói tin cho tỷ lệ chính xác 100%, chứng minh tính tin cậy của giao thức.

Phần mềm giao diện C# hoạt động tốt, hỗ trợ đầy đủ chức năng: kết nối COM, đọc dữ liệu thời gian thực, kiểm tra gói tin, hiển thị thông số và thông báo trạng thái.

1.2. Về mặt thực nghiệm

Tín hiệu UART giữa PC – mạch cách ly – RP2040 và giữa RP2040 – US-100 đều sạch, biên độ đúng chuẩn logic TTL, không xuất hiện méo xung hoặc nhiễu mức cao.

Kết quả đo khoảng cách cho sai số rất thấp (0–1% tùy vị trí đo), đáp ứng yêu cầu của đề tài và phù hợp với thực tế sử dụng.

Hệ thống vận hành ổn định trong thời gian dài, không xảy ra mất gói tin, CRC sai hoặc treo bộ xử lý.

1.3. Những điểm mới của luận văn

Qua quá trình thực hiện, luận văn mang đến các đóng góp sau:

Triển khai bộ đôi Software-UART hoạt động song song trên RP2040, phù hợp cho ứng dụng cần hai cổng UART nhưng không đủ UART phần cứng.

Chứng minh hiệu quả của kiến trúc PCB tách GND trong hệ thống đo lường dùng US-100, một hướng mà nhiều đồ án trước chưa đề cập sâu.

Hoàn thiện quy trình đo – xử lý – hiển thị theo chuẩn giao thức riêng (Frame + CRC), là cơ sở để phát triển các hệ thống truyền dữ liệu tin cậy.

Những kết quả đạt được cho thấy mô hình có tính ứng dụng cao trong đo lường khoảng cách, mực chất lỏng, hoặc các hệ thống robot và IoT.

2. HẠN CHẾ CỦA ĐỀ TÀI

Mặc dù đạt được các mục tiêu quan trọng, đề tài vẫn còn một số hạn chế:

Phạm vi đo giới hạn (5–30 cm) theo yêu cầu đề bài, chưa khai thác hết khả năng đo xa của US-100 (lên đến 450 cm).

Chưa đánh giá trong môi trường có nhiễu mạnh, nhiệt độ cao, gió hoặc bề mặt hấp thụ âm – những yếu tố ảnh hưởng đến độ chính xác của cảm biến.

Thuật toán lọc chỉ dừng ở mức trung bình trượt, chưa áp dụng các bộ lọc thích nghi hoặc Kalman Filter để tăng khả năng dự đoán.

Giao diện phần mềm còn đơn giản, chưa có biểu đồ theo thời gian thực hoặc chức năng lưu trữ và phân tích dữ liệu.

Chưa tích hợp hệ thống vỏ hộp cơ khí, chưa đánh giá tác động rung động và môi trường.

3. KIẾN NGHỊ

Dựa trên kết quả đạt được và những hạn chế còn tồn tại, nhóm đề xuất một số hướng phát triển như sau:

3.1. Hướng phát triển hệ thống

Mở rộng dải đo lên đến 450 cm, đồng thời cải tiến thuật toán để giảm sai số ở dải xa.

Tích hợp cảm biến nhiệt độ để bù vận tốc âm theo môi trường, tăng độ chính xác.

Nghiên cứu áp dụng thuật toán Kalman Filter hoặc Median Filter nhằm nâng cao chất lượng dữ liệu đầu ra.

Hoàn thiện cơ khí, thiết kế vỏ bảo vệ để đánh giá khả năng hoạt động ngoài thực địa.

3.2. Hướng phát triển phần mềm

Bổ sung biểu đồ theo thời gian thực, lưu dữ liệu vào file Excel/CSV.

Tích hợp tính năng cảnh báo trực quan (âm thanh, màu sắc).

Xây dựng module phân tích dữ liệu đo để ứng dụng trong IoT hoặc robot tự động.

3.3. Hướng mở rộng nghiên cứu

So sánh US-100 với các cảm biến khác như HC-SR04, VL53L0X, Lidar mini.

Tích hợp hệ thống vào robot tránh vật cản hoặc mô hình giám sát mức chất lỏng.

Nghiên cứu thiết kế một module đo khoảng cách thương mại hóa với độ ổn định cao hơn.

4. KẾT LUẬN CHUNG

Đề tài đã đạt được mục tiêu xây dựng một hệ thống đo khoảng cách có độ chính xác, độ ổn định và tính ứng dụng cao. Việc kết hợp đúng đắn giữa phần cứng, firmware và phần mềm đã tạo thành một mô hình hoàn chỉnh, có thể sử dụng trực tiếp trong thực tế hoặc mở rộng thành các sản phẩm công nghệ khác. Những kết quả này khẳng định tính khả thi và giá trị của đề tài trong lĩnh vực điện tử – đo lường – điều khiển.

TÀI LIỆU THAM KHẢO

- [1] Raspberry Pi Foundation, "RP2040 Datasheet"
- [2] Raspberry Pi Foundation, "Raspberry Pi Pico C/C++ SDK"
- [3] "US-100 Ultrasonic Sensor Module Datasheet"
- [4] Vishay Intertechnology, "6N137, High-Speed Optocoupler"
- [5] WCH, "CH340 Datasheet"
- [6] IPC-Association Connecting Electronics Industries, "IPC-2221A..."
- [7] Atmel Corporation, "AVR305: Software UART"
- [8] J. W. Valvano, "Digital Filtering in Embedded Systems"
- [9] "Introduction to State Machines in C/C++"
- [10] E. F. Philhower, III, "Arduino-Pico..."
- [11] Microsoft, "SerialPort Class (System.IO.Ports)"
- [12] S. Barman, "AMS1117 DC 5V Regulator Module", [Online].
- [13] Mạch chuyển USB UART CH340G, "Mạch chuyển USB UART CH340G,"
- [14] Syspcb, "Introduction of PCB design clearance requirements - SYS Technology Co., Ltd.," *SYS Technology Co., Ltd.*, Apr. 02, 2025.
- [15] "Polygons & Copper Regions," *Altium Documentation*, 2024.

PHỤ LỤC A

Đề tài: Đo khoảng cách bằng US-100 và RP2040

Firmware cho Vi điều khiển Raspberry Pi Pico

Tác giả: Tô Minh Cường

Ngôn ngữ: C++ (Trên nền tảng Arduino)

CHỨC NĂNG CHÍNH:

1. Sử dụng 2 class Software UART tùy chỉnh:

PicoSoftUART (dùng ngắt timer) cho Cảm biến US-100 trên chân 0, 1.

SoftUART_UI (dùng delay) cho Giao diện C# trên chân 16, 17 (qua bo cách ly).

Tổ chức theo State Machine (IDLE, READ, TEST).

Áp dụng bộ lọc trung bình trượt (10 mẫu) để lọc nhiễu cảm biến.

Đóng gói dữ liệu gửi lên C# bằng giao thức Frame (với CRC checksum).

. Điều khiển LED onboard (GPIO 25) để cảnh báo khi vật cản < 10cm.

Code:

<pre>#include <Arduino.h> #include "pico/time.h" // == PHẦN 1A: THƯ VIỆN PicoSoftUART (CHO CẢM BIẾN US- 100) ===== //Sử dụng ngắt timer (repeating_timer) để đạt độ chính xác cao == #define SU_RX_BUFFER_SIZE 64 #define SU_TX_BUFFER_SIZE 32</pre>	<pre>// == PHẦN 2: CÀI ĐẶT VÀ KHAI BÁO BIẾN TOÀN CỤC #define LED_PIN 25 // LED Onboard của Pico // Khởi tạo 2 đối tượng Software UART PicoSoftUART sensorUART(0, 1); // TX=0, RX=1 (Nối với US-100) SoftUART_UI uiUART(16, 17); // TX=16, RX=17 (Nối với bo cách ly)</pre>
---	--

<pre> class PicoSoftUART { public: PicoSoftUART(uint8_t txPin, uint8_t rxPin); void begin(long speed); int read(); size_t write(uint8_t byte); int available(); void print(String s); void println(String s); String readStringUntil(char terminator); void handleTimerInterrupt(); private: void txProcess(); void rxProcess(); uint8_t _txPin, _rxPin; long _bitPeriod; uint8_t _rx_buffer[SU_RX_BUFFER_SIZE]; volatile uint8_t _rx_buffer_head, _rx_buffer_tail; </pre>	<pre> // Định nghĩa các trạng thái của máy trạng thái chính enum Mode { Mode currentMode = IDLE; // Cấu hình cho bộ lọc trung bình trượt #define NUM_SAMPLES 10 float samples[NUM_SAMPLES]; int sampleCounter = 0; unsigned long lastSampleTime = 0; // Các trạng thái con cho chế độ READ (để chạy non-blocking) enum ReadState { READ_IDLE, READ_WAITING_FOR_SENSOR }; ReadState currentReadState = READ_IDLE; unsigned long readStateTimer = 0; // == PHẦN 3: HÀM HỖ TRỢ (CRC & TẠO FRAME) ===== * Tính CRC 8-bit đơn giản (Checksum) * @param s Chuỗi payload </pre>
--	--

<pre> uint8_t _tx_buffer[SU_TX_BUFFER_SIZE]; volatile uint8_t _tx_buffer_head, _tx_buffer_tail; volatile uint8_t _tx_bit_counter, _tx_byte, _rx_bit_counter, _rx_byte, _sample_counter, _rx_bit_offset; volatile bool _rx_timing_flag; }; // Con trỏ instance toàn cục để callback của timer có thể truy cập static PicoSoftUART* p_suart_instance = nullptr; // Hàm callback cho ngắt timer bool repeating_timer_callback(struct repeating_timer *t) { if (p_suart_instance) { p_suart_instance->handleTimerInterrupt(); } return true; } PicoSoftUART::PicoSoftUART(uint8_t txPin, uint8_t rxPin) { </pre>	<pre> * @return Giá trị CRC 8-bit uint8_t crc(const char *s) { uint16_t sum = 0; for (int i = 0; s[i] != '\0'; i++) { sum += (uint8_t)s[i]; } return (uint8_t)(sum % 256); } * Tạo một gói tin (frame) chuẩn hóa để gửi lên C# * Định dạng: @DIST_[value]_[CRC] * @param val Giá trị khoảng cách (float) * @return Chuỗi String đã đóng gói */ String make_frame(float val) { char core_buffer[32]; snprintf(core_buffer, 32, "@DIST_%.1f_#", val); String frame = String(core_buffer); frame += String(crc(core_buffer)); // Thêm CRC vào cuối return frame; </pre>
---	---

<pre> _txPin = txPin; _rxPin = rxPin; p_suart_instance = this; // Gán instance này cho con trỏ toàn cục } void PicoSoftUART::begin(long speed) { _bitPeriod = 1000000 / speed; pinMode(_txPin, OUTPUT); digitalWrite(_txPin, HIGH); pinMode(_rxPin, INPUT_PULLUP); _rx_buffer_head = 0; _rx_buffer_tail = 0; _tx_buffer_head = 0; _tx_buffer_tail = 0; _tx_bit_counter = 0; _rx_bit_counter = 0; _sample_counter = 0; _rx_timing_flag = false // Cấu hình timer ngắt 5 lần mỗi bit (oversampling) long timer_interval = -(_bitPeriod / 5); </pre>	<pre> } // == PHẦN 4: CHƯƠNG TRÌNH CHÍNH void setup() { Serial.begin(115220); // UART-USB vật lý để debug pinMode(LED_PIN, OUTPUT); digitalWrite(LED_PIN, LOW); delay(2000); // Chờ USB ổn định Serial.println("--- Firmware đã sản sang (Phiên bản 3 - LED & CRC) ---"); // Khởi tạo UART cho giao diện C# uiUART.begin(9600); Serial.println("UART Giao diện (Software) đã khởi tạo trên chân 16, 17."); // Khởi tạo UART cho cảm biến US- 100 sensorUART.begin(9600); Serial.println("UART Cảm biến (Software) đã khởi tạo trên chân 0, 1."); // Gửi tín hiệu sẵn sàng cho C# </pre>
---	---

<pre> struct repeating_timer *timer = new struct repeating_timer; add_repeating_timer_us(timer_interval, repeating_timer_callback, NULL, timer); } void PicoSoftUART::handleTimerInterrupt() { rxProcess(); // Luôn xử lý nhận if (_sample_counter == 0) { txProcess(); // Xử lý truyền tại 1 điểm cố định trong chu kỳ } } void PicoSoftUART::txProcess() { if (_tx_bit_counter > 0) { _tx_bit_counter--; if (_tx_bit_counter == 10) { // Start bit digitalWrite(_txPin, LOW); } else if (_tx_bit_counter > 1) { // Data bits </pre>	<pre> uiUART.println("FIRMWARE_READ Y"); } * Hàm state machine cho chế độ READ (Non-blocking) * Được gọi liên tục trong loop() khi currentMode == READ void run_read_mode_non_blocking() { switch (currentReadState) { case READ_IDLE: // Gửi lệnh yêu cầu cảm biến đo mỗi 500ms if (millis() - lastSampleTime >= 500) { lastSampleTime = millis(); sensorUART.write(0x55); // Lệnh yêu cầu đo của US-100 currentReadState = READ_WAITING_FOR_SENSOR; readStateTimer = millis(); // Bắt đầu đếm giờ chờ } break; </pre>
---	--

<pre> digitalWrite(_txPin, (_tx_byte >> (9 - _tx_bit_counter)) & 1); } else { // Stop bit digitalWrite(_txPin, HIGH); } } else if (_tx_buffer_head != _tx_buffer_tail) { _tx_byte = _tx_buffer[_tx_buffer_tail]; _tx_buffer_tail = (_tx_buffer_tail + 1) % SU_TX_BUFFER_SIZE; _tx_bit_counter = 11; // 1 start + 8 data + 2 stop (an toàn) } } void PicoSoftUART::rxProcess() { bool pin_state = digitalRead(_rxPin) // Phát hiện start bit if (!_rx_timing_flag && !pin_state) { _rx_bit_offset = (_sample_counter + 2) % 5; // Lấy mẫu ở giữa bit _rx_timing_flag = true; </pre>	<pre> case READ_WAITING_FOR_SENSOR: // Hủy nếu chờ quá 100ms mà không có dữ liệu if (millis() - readStateTimer > 100) { Serial.println("[READ_SM] Sensor timeout."); currentReadState = READ_IDLE; return; } bool validSample = false; float distance = 0.0; // Nếu có đủ 2 byte từ cảm biến if (sensorUART.available() >= 2) { uint8_t high_byte = sensorUART.read(); uint8_t low_byte = sensorUART.read(); distance = ((high_byte << 8) low_byte) / 10.0; // US-100 trả về mm, đổi sang cm </pre>
---	--

<pre> _rx_bit_counter = 10; // 1 start + 8 data + 1 stop _rx_byte = 0; } // Lấy mẫu if (_rx_timing_flag && (_sample_counter == _rx_bit_offset)) { if (_rx_bit_counter > 0) { _rx_bit_counter--; if (_rx_bit_counter > 0) { // 8 data bits _rx_byte >>= 1; if (pin_state) { _rx_byte = 0x80; } } else { // Stop bit if (pin_state) { // Stop bit phải là HIGH uint8_t next_head = (_rx_buffer_head + 1) % SU_RX_BUFFER_SIZE; if (next_head != _rx_buffer_tail) { </pre>	<pre> // Kiểm tra giá trị hợp lệ if (distance > 2 && distance < 450) { Serial.print("[READ_SM] Doc lan " + String(sampleCounter + 1) + ": " + String(distance, 1) + " cm\n"); validSample = true; } } // Xóa bộ đệm (flush) để tránh dữ liệu cũ while (sensorUART.available()) { sensorUART.read(); } if (validSample) { samples[sampleCounter++] = distance; // Khi thu thập đủ 10 mẫu if (sampleCounter >= NUM_SAMPLES) { float sum = 0.0; for (int i = 0; i < NUM_SAMPLES; i++) { </pre>
--	--

<pre> _rx_buffer[_rx_buffer_head] = _rx_byte; _rx_buffer_head = next_head; } } _rx_timing_flag = false; // Sẵn sàng nhận byte tiếp theo } } } _sample_counter = (_sample_counter + 1) % 5; } int PicoSoftUART::available() { return (_rx_buffer_head + SU_RX_BUFFER_SIZE - _rx_buffer_tail) % SU_RX_BUFFER_SIZE; } int PicoSoftUART::read() { if (_rx_buffer_head == _rx_buffer_tail) { </pre>	<pre> sum += samples[i]; } float avg = sum / NUM_SAMPLES; // Logic bật LED nếu < 10cm if (avg < 10.0) { digitalWrite(LED_PIN, HIGH); } else { digitalWrite(LED_PIN, LOW); } // Đóng gói và gửi lên C# String frame_to_send = make_frame(avg); uiUART.println(frame_to_send); Serial.println("[READ_SM] Da gui frame: " + frame_to_send); sampleCounter = 0; // Reset bộ đếm mẫu } </pre>
--	--

<pre> return -1; // Buffer rỗng } uint8_t c = _rx_buffer[_rx_buffer_tail]; _rx_buffer_tail = (_rx_buffer_tail + 1) % SU_RX_BUFFER_SIZE; return c; } size_t PicoSoftUART::write(uint8_t byte) { uint8_t next_head = (_tx_buffer_head + 1) % SU_TX_BUFFER_SIZE; if (next_head == _tx_buffer_tail) { return 0; // Buffer đầy } _tx_buffer[_tx_buffer_head] = byte; _tx_buffer_head = next_head; return 1; } void PicoSoftUART::print(String s) { for (int i = 0; i < s.length(); i++) { write(s[i]); </pre>	<pre> } currentReadState = READ_IDLE; // Quay về chờ break; } } * Hàm blocking cho chế độ TEST * Chạy 1 vòng lặp riêng và chỉ thoát khi có lệnh void run_dedicated_test_mode() { Serial.println("Entering dedicated TEST mode loop (with CRC check)..."); while (currentMode == TEST) { if (uiUART.available()) { String line = uiUART.readStringUntil('\n'); line.trim(); if (line.equalsIgnoreCase("STOP")) { currentMode = IDLE; Serial.println("Nhan lenh STOP -> Thoat che do TEST."); </pre>
---	--

<pre> } } void PicoSoftUART::println(String s) { print(s); write('\n'); } String PicoSoftUART::readStringUntil(char terminator) { String ret; int c = read(); while (c >= 0 && c != terminator) { ret += (char)c; c = read(); } return ret; } // == PHẦN 1B: THƯ VIỆN SoftUART_UI (CHO GIAO DIỆN C#) // == Sử dụng delayMicroseconds (blocking) đơn giản ===== class SoftUART_UI { </pre>	<pre> uiUART.println("ACK_IDLE_MODE"); } else if (line.equalsIgnoreCase("READ")) { currentMode = READ; Serial.println("Nhan lenh READ -> Thoat che do TEST."); uiUART.println("ACK_READ_MODE "); sampleCounter = 0; currentReadState = READ_IDLE; lastSampleTime = millis() - 500; // Đảm bảo đọc ngay } else if (line.length() > 0) { // Logic kiểm tra CRC int separatorIndex = line.lastIndexOf('#'); if (separatorIndex != -1) { String payload = line.substring(0, separatorIndex); String crcStr = line.substring(separatorIndex + 1); </pre>
---	--

<pre> private: uint8_t _txPin, _rxPin; long _bitPeriod; public: SoftUART_UI(uint8_t txPin, uint8_t rxPin) { _txPin = txPin; _rxPin = rxPin; } void begin(long speed) { _bitPeriod = 1000000 / speed; pinMode(_txPin, OUTPUT); digitalWrite(_txPin, HIGH); pinMode(_rxPin, INPUT_PULLUP); } size_t write(uint8_t byte) { // Start bit digitalWrite(_txPin, LOW); delayMicroseconds(_bitPeriod); // 8 Data bits for (int i = 0; i < 8; i++) { </pre>	<pre> uint8_t receivedCrc = crcStr.toInt(); // Tính toán CRC từ payload uint8_t calculatedCrc = crc(payload.c_str()); // So sánh và chỉ echo nếu CRC hợp lệ if (receivedCrc == calculatedCrc) { uiUART.println(line); // Echo lại toàn bộ gói tin Serial.println("[TEST_CRC_OK] Echo: " + line); } else { Serial.println("[TEST_CRC_FAIL] Goi tin bi loi, bo qua. Nhan: " + line); } } else { Serial.println("[TEST_NO_CRC] Goi tin khong co CRC, bo qua: " + line); } </pre>
--	---

<pre> if (byte & 1) { digitalWrite(_txPin, HIGH); } else { digitalWrite(_txPin, LOW); } byte >>= 1; delayMicroseconds(_bitPeriod); } // Stop bit digitalWrite(_txPin, HIGH); delayMicroseconds(_bitPeriod); return 1; } bool available() { return (digitalRead(_rxPin) == LOW); // Chỉ kiểm tra start bit } int read() { // Chờ start bit (blocking) while (digitalRead(_rxPin) == HIGH) { </pre>	<pre> } } yield(); // Cho phép các tác vụ nền } Serial.println("Exited dedicated TEST mode loop."); } * Vòng lặp chính, điều phối State Machine void loop() { // 1. Luôn kiểm tra lệnh từ UI (Giao diện C#) if (uiUART.available()) { String command = uiUART.readStringUntil('\n'); command.trim(); if (command.equalsIgnoreCase("READ")) { currentMode = READ; sampleCounter = 0; currentReadState = READ_IDLE; </pre>
---	--

<pre> yield(); // Cho phép các tác vụ nền (nếu có) } // Căn giữa thời điểm lấy mẫu delayMicroseconds(_bitPeriod * 1.5); uint8_t data = 0; for (int i = 0; i < 8; i++) { if (digitalRead(_rxPin) == HIGH) { data = (1 << i); } delayMicroseconds(_bitPeriod); } return data; } void print(String s) { for (int i = 0; i < s.length(); i++) { write(s[i]); } } void println(String s) { </pre>	<pre> lastSampleTime = millis() - 500; // Đảm bảo lần đọc đầu tiên diễn ra ngay Serial.println("Nhan lenh READ."); uiUART.println("ACK_READ_MODE "); } else if (command.equalsIgnoreCase("TEST")) { currentMode = TEST; digitalWrite(LED_PIN, LOW); // Tắt LED khi vào test Serial.println("Nhan lenh TEST."); uiUART.println("ACK_TEST_MODE"); } else if (command.equalsIgnoreCase("STOP")) { currentMode = IDLE; digitalWrite(LED_PIN, LOW); // Tắt LED khi nghỉ </pre>
--	--

<pre> print(s); write('\n'); } String readStringUntil(char terminator) { String ret; char c; while (1) { if (available()) { c = (char)read(); // Hàm read này là blocking if (c == terminator) { break;} ret += c; } else { yield(); } } return ret;} }; IDLE, READ, TEST };</pre>	<pre> Serial.println("Nhan lenh STOP."); uiUART.println("ACK_IDLE_MODE"); } } // 2. Chạy State Machine dựa trên chế độ hiện tại if (currentMode == READ) { run_read_mode_non_blocking(); } else if (currentMode == TEST) { run_dedicated_test_mode(); // Hàm này chứa vòng lặp riêng } else { // currentMode == IDLE yield(); // Không làm gì cả, nghỉ } }</pre>
--	---

PHỤ LỤC B

Đề tài: Đo khoảng cách bằng US-100 và RP2040

Phần mềm giao diện (Software) – Chương trình C# WinForms

Tác giả: Nguyễn Công Luận

Ngôn ngữ: C# (trên nền tảng Visual Studio 2022)

CHỨC NĂNG CHÍNH

1. Quản lý cổng COM và tự động kết nối lại
2. Giao tiếp UART với Raspberry Pi Pico
3. Gửi lệnh điều khiển sang Pico
4. Chế độ Test truyền thông (TEST Mode – 1000 frame CRC)
5. Giải mã frame dữ liệu khoảng cách
6. Hiển thị trực quan dữ liệu và trạng thái hệ thống

CODE (SOFTWARE – C#)

<pre>using System; using System.Diagnostics; using System.Drawing; using System.IO.Ports; using System.Linq; using System.Threading.Tasks; using System.Windows.Forms; using Timer = System.Windows.Forms.Timer;</pre>	<pre> btnConnect.Text = "Ngắt kết nối"; btnConnect.BackColor = Color.OrangeRed; lblStatus.Text = \$"Đã kết nối tới {serialPort.PortName}. Chờ lệnh..."; lblStatus.ForeColor = Color.DarkGreen; }</pre>
---	---

<pre> namespace US100_RP2040_App { public partial class Form1 : Form { private SerialPort serialPort; private bool isTesting = false; private int sentCount = 0; private int receivedCount = 0; private string lastPortName = null; private Timer portMonitorTimer; private string[] previousPorts = Array.Empty<string>(); // Các control được tạo bằng code private ComboBox cboPorts; private Button btnConnect; private bool hasEverConnected = false; public Form1() { </pre>	<pre> catch (Exception ex) { MessageBox.Show(\$"Không thể kết nối: {ex.Message}", "Lỗi kết nối", MessageBoxButtons.OK, MessageBoxIcon.Error); } } private void btnDo_Click(object sender, EventArgs e) { if (!serialPort.IsOpen) { MessageBox.Show("Vui lòng kết nối tới cổng COM trước.", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning); return; } isTesting = false; string command = "READ"; </pre>
--	---

<pre> InitializeComponent(); // Bắt buộc: Gọi giao diện từ file Designer InitializeCustomComponents(); // Thêm các control tự tạo (ComboBox, nút Connect) LoadSerialPorts(); serialPort = new SerialPort(); serialPort.DataReceived += SerialPort_DataReceived; // --- Khởi tạo Timer để theo dõi cổng COM --- portMonitorTimer = new Timer(); portMonitorTimer.Interval = 1000; // kiểm tra mỗi 1 giây portMonitorTimer.Tick += PortMonitorTimer_Tick; portMonitorTimer.Start(); } /// <summary> /// Hàm này khởi tạo các control không có trong file Designer gốc. /// </summary> </pre>	<pre> serialPort.WriteLine(command); txtResponse.AppendText(\$"[GỬI] -> {command}\r\n"); lblStatus.Text = "Đã gửi lệnh READ. Đang chờ dữ liệu khoảng cách..."; } private async void btnTest_Click(object sender, EventArgs e) { if (!serialPort.IsOpen) { MessageBox.Show("Vui lòng kết nối tới cổng COM trước.", "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning); return; } SetButtonsEnabled(false); isTesting = true; sentCount = 0; </pre>
---	---

<pre> private void InitializeCustomComponents() { // --- ComboBox để chọn cổng COM --- this.cboPorts = new ComboBox(); this.cboPorts.Location = new Point(12, 25); // Điều chỉnh vị trí cho đẹp this.cboPorts.Size = new Size(121, 21); this.cboPorts.DropDownStyle = ComboBoxStyle.DropDownList; this.Controls.Add(this.cboPorts); // --- Nút Kết nối / Ngắt kết nối this.btnConnect = new Button(); this.btnConnect.Location = new Point(139, 23); // Điều chỉnh vị trí cho đẹp this.btnConnect.Size = new Size(90, 25); </pre>	<pre> receivedCount = 0; txtFirmware.Clear(); txtResponse.Clear(); serialPort.WriteLine("TEST"); txtResponse.AppendText("[GỬI] -> TEST\r\n"); lblStatus.Text = "Bắt đầu bài test... Đang gửi 1000 gói tin có CRC."; await Task.Delay(200); for (int i = 0; i < 1000; i++) { string payload = \$"hello_{i + 1}"; int crcValue = Crc(payload); string packet = \$" {payload}#{crcValue}"; serialPort.WriteLine(packet); sentCount++; lblSendRate.Text = \$"Đã gửi: {sentCount}"; txtResponse.AppendText(\$"[GỬI] -> {packet}\r\n"); </pre>
--	--

<pre> this.btnConnect.Text = "Kết nối"; this.btnConnect.Font = new Font("Segoe UI", 9F, FontStyle.Bold); this.btnConnect.BackColor = Color.DodgerBlue; this.btnConnect.ForeColor = Color.White; this.btnConnect.Click += new EventHandler(this.btnConnect_Click); this.Controls.Add(this.btnConnect); } private void LoadSerialPorts() { cboPorts.Items.Clear(); string[] ports = SerialPort.GetPortNames(); if (ports.Length > 0) { cboPorts.Items.AddRange(ports); cboPorts.SelectedIndex = 0; } </pre>	<pre> await Task.Delay(35); } txtResponse.AppendText(\$"--- Đã gửi xong {sentCount} gói tin --- \r\n"); lblStatus.Text = "Test hoàn tất. Đang chờ các gói tin phản hồi cuối cùng..."; await Task.Delay(2000); isTesting = false; SetButtonsEnabled(true); lblStatus.Text = "Bài test đã kết thúc."; UpdateSuccessRate(); } private void btnRestart_Click(object sender, EventArgs e) { if (serialPort.IsOpen) { isTesting = false; </pre>
---	---

<pre> else { lblStatus.Text = "Không tìm thấy cổng COM nào."; lblStatus.ForeColor = Color.Red; } } // Theo dõi thay đổi cổng COM và tự động kết nối lại private void PortMonitorTimer_Tick(object sender, EventArgs e) { string[] currentPorts = SerialPort.GetPortNames(); // Cập nhật danh sách COM nếu có thay đổi if (!currentPorts.SequenceEqual(previous Ports)) { cboPorts.Items.Clear(); </pre>	<pre> serialPort.WriteLine("STOP"); txtResponse.AppendText("[GỬI] -> STOP\r\n"); } txtFirmware.Clear(); txtResponse.Clear(); lblDistance.Text = "Khoảng cách: -- cm"; lblReceived.Text = "Đã nhận: 0"; lblSendRate.Text = "Đã gửi: 0"; lblSuccessRate.Text = "Tỉ lệ gửi đúng: --"; sentCount = 0; receivedCount = 0; lblStatus.Text = "Đã reset. Chờ lệnh..."; } private void btnExit_Click(object sender, EventArgs e) { Application.Exit(); </pre>
--	--

<pre> cboPorts.Items.AddRange(currentPorts); if (currentPorts.Length > 0) { if (cboPorts.SelectedItem == null) cboPorts.SelectedIndex = 0; lblStatus.Text = \$" Phát hiện {currentPorts.Length} cổng COM: {string.Join(", ", currentPorts)}"; lblStatus.ForeColor = Color.DarkBlue; } else { lblStatus.Text = " Không phát hiện thiết bị nào."; lblStatus.ForeColor = Color.Red; } previousPorts = currentPorts; </pre>	<pre> } private void SetButtonsEnabled(bool enabled) { btnDo.Enabled = enabled; btnTest.Enabled = enabled; btnRestart.Enabled = enabled; btnConnect.Enabled = enabled; } #endregion #region Serial Port and Data Handling private int Crc(string s) { int sum = 0; foreach (char c in s) { sum += (byte)c; } return sum % 256; } private void SerialPort_DataReceived(object sender, SerialDataReceivedEventArgs e) </pre>
--	--

<pre> } // Nếu đang kết nối mà cổng bị mất → đóng lại if (serialPort.IsOpen && !currentPorts.Contains(serialPort.Port Name)) { serialPort.Close(); lblStatus.Text = " Mất kết nối - đang chờ thiết bị..."; lblStatus.ForeColor = Color.OrangeRed; btnConnect.Text = "Kết nối"; btnConnect.BackColor = Color.DodgerBlue; return; } // Nếu chưa kết nối mà có COM mới → tự động kết nối // Nếu chưa kết nối mà có COM mới → tự động kết nối (kể cả lần đầu) </pre>	<pre> { try { while (serialPort.BytesToRead > 0) { string data = serialPort.ReadLine().Trim(); this.Invoke(new Action(() => ProcessReceivedData(data))); } } catch (Exception ex) { Debug.WriteLine(\$"Lỗi khi nhận dữ liệu: {ex.Message}"); } } private void ProcessReceivedData(string data) { if (string.IsNullOrEmpty(data)) return; </pre>
--	--

<pre> // Tự động kết nối lại chỉ khi trước đó đã kết nối ít nhất 1 lần if (!serialPort.IsOpen && hasEverConnected && currentPorts.Length > 0) { string targetPort = lastPortName; // Nếu cổng trước đó bị mất, tìm lại trong danh sách mới if (string.IsNullOrEmpty(targetPort) !currentPorts.Contains(targetPort)) targetPort = currentPorts[0]; try { serialPort.PortName = targetPort; serialPort.BaudRate = 9600; serialPort.Open(); lastPortName = serialPort.PortName; </pre>	<pre> txtFirmware.AppendText(\$"[NHẬN] <- {data}\r\n"); if (isTesting) { if (data.StartsWith("hello_")) { receivedCount++; lblReceived.Text = \$"Đã nhận: {receivedCount}"; UpdateSuccessRate(); } } else { if (data.StartsWith("@DIST_") && data.Contains("_#")) { try { </pre>
---	---

<pre> lblStatus.Text = \$" Đã tự động kết nối lại: {serialPort.PortName}"; lblStatus.ForeColor = Color.DarkGreen; btnConnect.Text = "Ngắt kết nối"; btnConnect.BackColor = Color.OrangeRed; cboPorts.SelectedItem = targetPort; } catch { lblStatus.Text = \$" Không thể mở lại {targetPort}."; lblStatus.ForeColor = Color.Red; } } } // ===== </pre>	<pre> int crcSeparatorIndex = data.LastIndexOf("_#"); string core = data.Substring(0, crcSeparatorIndex + 2); int receivedCrc = int.Parse(data.Substring(crcSeparatorInd ex + 2)); if (receivedCrc == Crc(core)) { string distanceStr = core.Replace("@DIST_", "").Replace("_#", ""); float distance = float.Parse(distanceStr, System.Globalization.CultureInfo.Invari antCulture); lblDistance.Text = \$"Khoảng cách: {distance:F1} cm"; lblStatus.Text = \$"Nhận thành công frame lúc {DateTime.Now:HH:mm:ss}"; lblStatus.ForeColor = Color.DarkGreen; </pre>
--	---

<pre> #region Event Handlers for Buttons private void btnConnect_Click(object sender, EventArgs e) { if (serialPort.IsOpen) { serialPort.Close(); btnConnect.Text = "Kết nối"; btnConnect.BackColor = Color.DodgerBlue; lblStatus.Text = "Đã ngắt kết nối."; lblStatus.ForeColor = Color.Red; } else { if (cboPorts.SelectedItem == null) { </pre>	<pre> } else { lblStatus.Text = \$"LỖI CRC!"; lblStatus.ForeColor = Color.Red; } } catch (Exception ex) { lblStatus.Text = \$"Lỗi khi xử lý frame: {ex.Message}"; } } } private void UpdateSuccessRate() { if (sentCount > 0) { </pre>
--	--

<pre> MessageBox.Show("Vui lòng chọn một cổng COM.", "Lỗi", MessageBoxButtons.OK, MessageBoxIcon.Error); return; } try { serialPort.PortName = cboPorts.SelectedItem.ToString(); serialPort.BaudRate = 9600; serialPort.Open(); hasEverConnected = true; lastPortName = serialPort.PortName; </pre>	<pre> double rate = ((double)receivedCount / sentCount) * 100.0; lblSuccessRate.Text = \$"Tỉ lệ gửi đúng: {rate:F1}%"; } } private void Form1_FormClosing(object sender, FormClosingEventArgs e) { if (serialPort.IsOpen) { serialPort.Close(); } } #endregion } } </pre>
--	---